

```
// File: miniprogram/packageMaps/pages/campus-map/campus-map.js
const campusMapDataService = require("../../../../services/campusMapDataService");
let campusData = campusMapDataService.getFallbackData();
const CAMPUS_TABS = [
  { key: "xianxi", label: "仙溪" },
  { key: "jiangwan", label: "江湾" },
  { key: "hebin", label: "河滨" },
];
const XIANXI_AREAS = [
  { key: "north", label: "北区" },
  { key: "south", label: "南区" },
];
const MAPS = {
  jiangwan: {
    key: "jiangwan",
    campus: "江湾校区",
    area: "江湾校区",
    title: "江湾校区",
    asset: "/packageMaps/assets/maps/campus-map-jiangwan.jpg",
  },
  xianxiNorth: {
    key: "xianxiNorth",
    campus: "仙溪校区",
    area: "北区",
    title: "仙溪校区北区",
    asset: "/packageMaps/assets/maps/campus-map-xianxi-north.jpg",
  },
  xianxiSouth: {
    key: "xianxiSouth",
    campus: "仙溪校区",
    area: "南区",
    title: "仙溪校区南区",
    asset: "/packageMaps/assets/maps/campus-map-xianxi-south.jpg",
  },
  hebin: {
    key: "hebin",
    campus: "河滨校区",
    area: "河滨校区",
    title: "河滨校区",
    asset: "/packageMaps/assets/maps/campus-map-hebin.jpg",
  },
};
function runtimeMapInfo(base) {
  const published = campusData && campusData.maps && campusData.maps[base.key] || {};
  const cloudbaseAsset = published.cdnUrl || published.cloudbaseUrl || "";
  const oracleAsset = published.fallbackUrl || published.oracleUrl || "";
  const packageAsset = base.asset;
  const asset = cloudbaseAsset || oracleAsset || packageAsset;
  return Object.assign({}, base, {
    title: published.title || base.title,
    asset,
    cloudbaseAsset,
    oracleAsset,
    packageAsset,
    assetSource: cloudbaseAsset ? "cloudbase" : (oracleAsset ? "oracle" : "package"),
    version: published.assetVersion || "",
  });
}
```

```

    hash: published.sha256 || "",
  });
}
function normalizeText(value) {
  return String(value || "").trim().replace(/\s+/g, "").toLowerCase();
}
function getPlaceSearchText(place) {
  return [
    place.id,
    place.campus,
    place.area,
    place.name,
    place.code,
    place.type,
  ]
    .concat(place.aliases || [])
    .map(normalizeText)
    .filter(Boolean)
    .join("|");
}
function getMapKey(campus, area) {
  if (campus === "江湾校区" || campus === "江湾") return "jiangwan";
  if (campus === "河滨校区" || campus === "河滨") return "hebin";
  if (campus === "仙溪校区" || campus === "仙溪") {
    return area === "南区" || area === "south" ? "xianxiSouth" : "xianxiNorth";
  }
  return "xianxiNorth";
}
function getCampusFromMapKey(mapKey) {
  if (mapKey === "hebin") return "hebin";
  if (mapKey === "xianxiNorth" || mapKey === "xianxiSouth") return "xianxi";
  return "jiangwan";
}
function withReviewStatus(place) {
  if (!place || typeof place !== "object") return place;
  return Object.assign({}, place, {
    reviewStatus: place.reviewStatus || (place.verified === true ? "verified" : "needs-review"),
  });
}
function placeMatchesMap(place, mapInfo) {
  return place && mapInfo &&
    place.campus === mapInfo.campus &&
    (place.area === mapInfo.area || place.type === "campus" || place.type === "area");
}
Page({
  data: {
    campusTabs: CAMPUS_TABS,
    xianxiAreas: XIANXI_AREAS,
    activeCampus: "xianxi",
    activeXianxiArea: "north",
    mapInfo: MAPS.xianxiNorth,
    places: [],
    query: "",
    results: [],
    searchState: "idle",
  },

```

```

    searchedQuery: "",
    searchError: "",
    selectedPlace: null,
    imageLoaded: false,
    imageError: false,
    previewVisible: false,
    previewImagePath: "",
    previewImageLoaded: false,
    previewImageError: false,
    previewScale: 1,
    note: campusData.note || "Q 版地图仅供校园位置参考，具体以学校现场指引为准。",
  },
  onLoad(options = {}) {
    let mapKey = getMapKey(options.campus || "", options.area || "");
    if (options.map && MAPS[options.map]) mapKey = options.map;
    const place = options.placeId ? this.findPlaceById(options.placeId) : null;
    if (place) mapKey = getMapKey(place.campus, place.area);
    const activeCampus = getCampusFromMapKey(mapKey);
    const activeXianxiArea = mapKey === "xianxiSouth" ? "south" : "north";
    this.setData({
      activeCampus,
      activeXianxiArea,
      query: options.q ? decodeURIComponent(options.q) : "",
    });
    this.updateMapData(mapKey, place);
    this.loadPublishedMapData(mapKey, place);
  },
  loadPublishedMapData(mapKey, selectedPlace) {
    campusMapDataService.loadPublishedMapData().then((data) => {
      campusData = data || campusData;
      const activeMapKey = this.data.mapInfo && this.data.mapInfo.key || mapKey || "xianxi
North";
      const nextPlace = selectedPlace && selectedPlace.id ? this.findPlaceById(selectedPla
ce.id) : this.data.selectedPlace;
      this.setData({
        note: campusData.note || this.data.note,
      });
      this.updateMapData(activeMapKey, nextPlace);
      if (this.data.searchState === "success" || this.data.searchState === "empty") {
        this.runSearch(this.data.searchedQuery || this.data.query || "");
      }
    });
  },
  findPlaceById(id) {
    const target = String(id || "");
    return withReviewStatus((campusData.places || []).find((place) => place.id === target)
|| null);
  },
  updateMapData(mapKey, selectedPlace) {
    const mapInfo = runtimeMapInfo(MAPS[mapKey] || MAPS.jiangwan);
    const places = (campusData.places || []).
      .filter((place) => placeMatchesMap(place, mapInfo))
      .map(withReviewStatus)
      .sort((left, right) => {
        const typeRank = { campus: 0, area: 1, teaching_building: 2, library: 3, canteen:
4 };

```

```
        return (typeRank[left.type] || 9) - (typeRank[right.type] || 9) ||
            String(left.code || left.name).localeCompare(String(right.code || right.name));
    });
    this.setData({
        mapInfo,
        places,
        selectedPlace: selectedPlace || null,
        imageLoaded: false,
        imageError: false,
    });
},
onCampusTap(event) {
    const campus = event.currentTarget.dataset.key;
    const mapKey = campus === "xianxi"
        ? (this.data.activeXianxiArea === "south" ? "xianxiSouth" : "xianxiNorth")
        : campus;
    this.setData({ activeCampus: campus });
    this.updateMapData(mapKey, null);
},
onAreaTap(event) {
    const area = event.currentTarget.dataset.key;
    const mapKey = area === "south" ? "xianxiSouth" : "xianxiNorth";
    this.setData({ activeCampus: "xianxi", activeXianxiArea: area });
    this.updateMapData(mapKey, null);
},
onSearchInput(event) {
    const query = event.detail.value || "";
    const patch = { query };
    if (!normalizeText(query)) {
        patch.results = [];
        patch.searchState = "idle";
        patch.searchedQuery = "";
        patch.searchError = "";
        patch.selectedPlace = null;
    }
    this.setData(patch);
},
onSearchConfirm(event) {
    const query = event.detail.value || this.data.query || "";
    this.runSearch(query);
},
onSearchTap() {
    this.runSearch(this.data.query || "");
},
clearSearch() {
    this.setData({
        query: "",
        results: [],
        searchState: "idle",
        searchedQuery: "",
        searchError: "",
        selectedPlace: null,
    });
},
runSearch(query, selectedPlace) {
    const q = normalizeText(query);
```

```

    if (!q) {
      this.setData({
        query: "",
        results: [],
        searchState: "idle",
        searchedQuery: "",
        searchError: "",
        selectedPlace: selectedPlace || null,
      });
      return;
    }
    this.setData({
      query,
      searchState: "loading",
      searchedQuery: query,
      searchError: "",
      results: [],
      selectedPlace: selectedPlace || null,
    });
    try {
      const results = (campusData.places || [])
        .map((place) => {
          const text = getPlaceSearchText(place);
          let score = 0;
          if (normalizeText(place.code) === q || normalizeText(place.name) === q) score =
100;
          else if ((place.aliases || []).some((alias) => normalizeText(alias) === q)) scor
e = 90;
          else if (text.includes(q)) score = 70;
          return { place, score };
        })
        .filter((item) => item.score > 0)
        .sort((left, right) => right.score - left.score || String(left.place.name).localeC
ompare(String(right.place.name)))
        .slice(0, 12)
        .map((item) => withReviewStatus(item.place));
      this.setData({
        results,
        searchState: results.length ? "success" : "empty",
        selectedPlace: selectedPlace || null,
      });
    } catch (error) {
      this.setData({
        results: [],
        searchState: "error",
        searchError: "搜索失败, 请稍后重试",
        selectedPlace: selectedPlace || null,
      });
    }
  },
  onPlaceTap(event) {
    const place = this.findPlaceById(event.currentTarget.dataset.id);
    if (place) this.focusPlace(place, true);
  },
  focusPlace(place, keepResults) {
    const mapKey = getMapKey(place.campus, place.area);

```

```

    this.setData({
      activeCampus: getCampusFromMapKey(mapKey),
      activeXianxiArea: mapKey === "xianxiSouth" ? "south" : "north",
    });
    this.updateMapData(mapKey, place);
    if (keepResults !== true && this.data.query) {
      this.setData({ results: this.data.results });
    }
  },
  previewMap() {
    const url = this.data.mapInfo && this.data.mapInfo.asset;
    if (!url) return;
    wx.getImageInfo({
      src: url,
      success: (info) => {
        const path = info && info.path || url;
        wx.previewImage({
          current: path,
          urls: [path],
          fail: () => this.openCustomPreview(path),
        });
      },
      fail: () => this.openCustomPreview(url),
    });
  },
  openCustomPreview(path) {
    this.setData({
      previewVisible: true,
      previewImagePath: path,
      previewImageLoaded: false,
      previewImageError: false,
      previewScale: 1,
    });
  },
  closeCustomPreview() {
    this.setData({
      previewVisible: false,
      previewImagePath: "",
      previewImageLoaded: false,
      previewImageError: false,
      previewScale: 1,
    });
  },
  resetCustomPreviewScale() {
    this.setData({ previewScale: 1 });
  },
  retryMapImage() {
    const key = this.data.mapInfo && this.data.mapInfo.key || "xianxiNorth";
    const mapInfo = runtimeMapInfo(MAPS[key] || MAPS.xianxiNorth);
    this.setData({ imageLoaded: false, imageError: false, mapInfo: Object.assign({}, mapInfo, { asset: "" }) }, () => {
      this.setData({ mapInfo });
    });
  },
  retryPreviewImage() {
    this.setData({

```

```

        previewImageLoaded: false,
        previewImageError: false,
    });
    const path = this.data.previewImagePath;
    this.setData({ previewImagePath: "" }, () => {
        this.setData({ previewImagePath: path });
    });
},
onPreviewImageLoad() {
    this.setData({ previewImageLoaded: true, previewImageError: false });
},
onPreviewImageError() {
    this.setData({ previewImageLoaded: false, previewImageError: true });
},
onMapImageLoad() {
    this.setData({ imageLoaded: true, imageError: false });
},
onMapImageError() {
    const mapInfo = Object.assign({}, this.data.mapInfo || {});
    if (mapInfo.assetSource === "cloudbase" && mapInfo.oracleAsset) {
        this.setData({
            imageLoaded: false,
            imageError: false,
            mapInfo: Object.assign({}, mapInfo, { asset: mapInfo.oracleAsset, assetSource: "or
acle" }),
        });
        return;
    }
    if (mapInfo.assetSource === "oracle" && mapInfo.packageAsset) {
        this.setData({
            imageLoaded: false,
            imageError: false,
            mapInfo: Object.assign({}, mapInfo, { asset: mapInfo.packageAsset, assetSource: "p
ackage" }),
        });
        return;
    }
    this.setData({ imageLoaded: false, imageError: true });
},
openSchoolQuery() {
    const place = this.data.selectedPlace;
    const q = place && (place.code || place.name) || this.data.query || "";
    if (!q) return;
    wx.navigateTo({
        url: `/pages/school/school?type=classroom&q=${encodeURIComponent(q)}&mapPlaceId=${en
codeURIComponent(place && place.id || "")}&mapCode=${encodeURIComponent(place && place.cod
e || q)}`,
    });
},
});
// File: miniprogram/pages/calendar/calendar.js
const {
    formatWeekRange,
    getTodayTeachingInfo,
} = require("../utils/week");
const teachingCalendarService = require("../services/teachingCalendarService");

```

```

function lastValidWeekNo(weeks) {
  const list = Array.isArray(weeks) ? weeks : [];
  for (let index = list.length - 1; index >= 0; index -= 1) {
    const weekNo = Number(list[index] && list[index].weekNo);
    if (weekNo > 0) return weekNo;
  }
  return 0;
}

Page({
  data: {
    title: "教学周历",
    loading: true,
    phaseText: "",
    currentWeek: 0,
    currentWeekText: "",
    targetWeekNo: 0,
    weeks: [],
    scrollTop: 0,
    showBackToCurrentWeek: false,
  },
  onShow() {
    this._calendarSeq = (this._calendarSeq || 0) + 1;
    this._calendarUserScrolled = false;
    this._autoCentering = false;
    this._lastAutoCenteredWeek = 0;
    this._lastTargetScrollTop = 0;
    const seq = this._calendarSeq;
    const immediate = teachingCalendarService.getImmediateActiveCalendar();
    const termConfig = immediate.termConfig || {};
    this.renderCalendar(immediate, termConfig, { forceCenter: true });
    teachingCalendarService.loadActiveTeachingCalendar({ pointerTimeout: 1800, calendarTim
eout: 2200 })
      .then((calendar) => {
        if (seq !== this._calendarSeq) return;
        this.renderCalendar(calendar, calendar.termConfig || termConfig, {
          forceCenter: false,
          allowChangedWeekCenter: true,
        });
      })
      .catch(() => {
        if (seq !== this._calendarSeq) return;
        // Never resurrect an archived built-in semester merely because the
        // active-term request timed out. Immediate runtime/last-good state is
        // authoritative; an unknown term stays explicitly unavailable.
        const fallback = teachingCalendarService.getImmediateActiveCalendar();
        this.renderCalendar(fallback, fallback.termConfig || termConfig, {
          forceCenter: false,
          allowChangedWeekCenter: true,
        });
      });
  },
  renderCalendar(calendar, termConfig, options = {}) {
    const effectiveConfig = calendar && calendar.termConfig || termConfig || {};
    const decorated = this.decorateWeeks(calendar, effectiveConfig);
    const todayInfo = getTodayTeachingInfo(new Date(), decorated, effectiveConfig);
    const phaseText = this.getPhaseText(todayInfo.termPhase);
  },

```

```

const targetWeekNo = this.resolveTargetWeekNo(todayInfo, decorated);
const previousTarget = this.data.targetWeekNo;
this.setData({
  loading: false,
  title: `${calendar.semesterText || effectiveConfig.semesterText || calendar.term ||
"当前学期"}教学周历`,
  currentWeek: todayInfo.isInTerm ? todayInfo.weekNo : 0,
  targetWeekNo,
  currentWeekText: todayInfo.isInTerm
    ? `${todayInfo.dateLabel} ${todayInfo.weekdayLabel} 第${todayInfo.weekNo}周`
    : (phaseText || "教学安排待维护"),
  phaseText,
  weeks: decorated,
});
const shouldCenter = targetWeekNo > 0 && (
  options.forceCenter ||
  (options.allowChangedWeekCenter && !this._calendarUserScrolled && targetWeekNo !== p
previousTarget)
);
if (shouldCenter) {
  this.centerTargetWeek({ force: options.forceCenter === true });
}
},
resolveTargetWeekNo(todayInfo, weeks) {
  if (todayInfo.termPhase === "in-term" && todayInfo.weekNo > 0) return todayInfo.weekNo
;
  if (todayInfo.termPhase === "before-term") return weeks.length ? Number(weeks[0].weekNo
o || 1) : 1;
  if (todayInfo.termPhase === "after-term") return lastValidWeekNo(weeks);
  return 0;
},
decorateWeeks(calendar, termConfig) {
  const weeks = calendar && Array.isArray(calendar.weeks) ? calendar.weeks : [];
  const effectiveConfig = calendar && calendar.termConfig || termConfig;
  const todayInfo = getTodayTeachingInfo(new Date(), weeks, effectiveConfig);
  return weeks.map((item) => Object.assign({}, item, {
    rangeText: item.startDate && item.endDate ? formatWeekRange(item.startDate, item.end
Date) : "",
    typeText: item.typeText || this.getTypeText(item.type),
    active: todayInfo.termPhase === "in-term" && item.weekNo === todayInfo.weekNo,
  }));
},
centerTargetWeek(options = {}) {
  const targetWeekNo = Number(this.data.targetWeekNo || 0);
  if (!targetWeekNo) return;
  if (!options.force && this._calendarUserScrolled) return;
  if (!options.force && this._lastAutoCenteredWeek === targetWeekNo) return;
  const runAfterRender = wx && typeof wx.nextTick === "function"
    ? wx.nextTick.bind(wx)
    : (callback) => setTimeout(callback, 0);
  runAfterRender(() => {
    if (!wx || typeof wx.createSelectorQuery !== "function") return;
    const query = wx.createSelectorQuery().in(this);
    query.select(".week-scroll").boundingClientRect();
    query.select(`#week-${targetWeekNo}`).boundingClientRect();
    query.exec((rects) => {

```

```

    const scrollRect = rects && rects[0];
    const cardRect = rects && rects[1];
    if (!scrollRect || !cardRect) return;
    const currentTop = Number(this.data.scrollTop || 0);
    const targetTop = Math.max(
        0,
        currentTop + cardRect.top - scrollRect.top - (scrollRect.height / 2) + (cardRect
.height / 2)
    );
    this._autoCentering = true;
    this._lastTargetScrollTop = targetTop;
    this._lastAutoCenteredWeek = targetWeekNo;
    this.setData({
        scrollTop: targetTop,
        showBackToCurrentWeek: false,
    });
    setTimeout(() => {
        this._autoCentering = false;
    }, 240);
    });
    });
},
onCalendarScroll(event) {
    const scrollTop = Number(event.detail && event.detail.scrollTop || 0);
    if (this._autoCentering) {
        this._lastScrollTop = scrollTop;
        return;
    }
    this._calendarUserScrolled = true;
    this._lastScrollTop = scrollTop;
    const targetWeekNo = Number(this.data.targetWeekNo || 0);
    const distance = Math.abs(scrollTop - Number(this._lastTargetScrollTop || 0));
    const shouldShow = targetWeekNo > 0 && distance > 260;
    if (shouldShow !== this.data.showBackToCurrentWeek) {
        this.setData({ showBackToCurrentWeek: shouldShow });
    }
},
backToCurrentWeek() {
    this._calendarUserScrolled = false;
    this.centerTargetWeek({ force: true });
},
getPhaseText(phase) {
    if (phase === "before-term") return "尚未开学";
    if (phase === "after-term") return "本学期已结束";
    if (phase === "unknown") return "学期日期待配置";
    return "";
},
getTypeText(type) {
    const map = {
        opening: "开学教学周",
        teaching: "正常教学周",
        holiday: "节假日/调休周",
        adjustment: "调整教学周",
        midterm: "期中教学检查",
        closing: "结课周",
        review: "复习周",
    };

```

```

    exam: "考试周",
    flexible: "机动周",
    pending: "教学安排待维护",
  };
  return map[type] || "正常教学周";
},
});
// File: miniprogram/pages/custom-courses/custom-courses.js
const customCourseService = require("../services/customCourseService");
const courseOverrideService = require("../services/courseOverrideService");
const { getBaseCoursesByClass } = require("../utils/course");
const { getCurrentScheduleTarget } = require("../utils/storage");
const weekdayOptions = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"];
const sectionOptions = Array.from({ length: 14 }, (_, index) => `第${index + 1}节`);
function getDefaultForm() {
  return {
    id: "",
    courseName: "",
    teacherName: "",
    classroom: "",
    weekdayIndex: 0,
    startSectionIndex: 0,
    endSectionIndex: 1,
    weekText: "1-16 周",
    color: "#3b82f6",
    note: "",
    enabled: true,
  };
}
function courseToForm(course) {
  const source = course || {};
  return {
    id: source.id || "",
    courseName: source.courseName || source.displayCourseName || source.canonicalCourseName || "",
    teacherName: source.teacherName || source.displayTeacherName || source.canonicalTeacherName || "",
    classroom: source.classroom || source.displayClassroom || source.canonicalClassroom || "",
    weekdayIndex: Math.max(0, Number(source.weekday || source.weekDay || 1) - 1),
    startSectionIndex: Math.max(0, Number(source.startSection || 1) - 1),
    endSectionIndex: Math.max(0, Number(source.endSection || 2) - 1),
    weekText: source.weekText || "1-16 周",
    color: source.color || "#3b82f6",
    note: source.note || source.remark || "",
    enabled: source.enabled !== false,
  };
}
function buildTargetLabel(target) {
  if (!target) {
    return "未绑定课表";
  }
  if (target.type === "teacher") {
    return `${target.name} 老师`;
  }
  if (target.type === "classroom") {

```

```

    return `${target.name} 教室`;
  }
  return target.name || target.className || "当前课表";
}
function filterSourceCourses(courses, query) {
  const keyword = String(query || "").trim().toLowerCase();
  if (!keyword) return courses;
  return courses.filter((item) => [item.courseName, item.teacherName, item.classroom]
    .some((value) => String(value || "").toLowerCase().includes(keyword)));
}
Page({
  data: {
    targetLabel: "未绑定课表",
    courses: [],
    sourceCourses: [],
    visibleSourceCourses: [],
    sourceQuery: "",
    sourceCount: 0,
    overrideCount: 0,
    formMode: "custom",
    sourceIndex: -1,
    formVisible: false,
    formTitle: "添加课程",
    weekdayOptions,
    sectionOptions,
    form: getDefaultForm(),
    importToolsCollapsed: true,
    predefinedColors: ["#3b82f6", "#10b981", "#f59e0b", "#ef4444", "#8b5cf6", "#ec4899", "#6b7280"]
  },
  onLoad() {
    const draft = customCourseService.takeCustomCourseDraft();
    const editDraft = courseOverrideService.takeEditDraft();
    this.loadCourses();
    if (editDraft) {
      const match = this.data.sourceCourses.find((item) =>
        (editDraft.sourceId && item.sourceId === editDraft.sourceId) ||
        item.signature === editDraft.signature);
      if (match) this.openSourceCourse(match.sourceIndex);
      else wx.showToast({ title: "原课程已更新, 请从列表重新选择", icon: "none" });
    } else if (draft) {
      this.setData({
        formVisible: true,
        formTitle: "复制为自定义课程",
        formMode: "custom",
        form: courseToForm(draft),
      });
    }
  },
  onShow() {
    this.loadCourses();
  },
  loadCourses() {
    const target = getCurrentScheduleTarget();
    const courses = customCourseService.getCustomCourses(target);
    const baseCourses = target ? getBaseCoursesByClass(target.name || target.className) :

```

```

[];
const effective = courseOverrideService.applyCourseOverrides(baseCourses, target);
const entries = courseOverrideService.getSourceEntries(baseCourses);
const sourceCourses = entries.map((entry) => {
  const course = effective[entry.index];
  const values = courseOverrideService.getSourceEditableValues(course);
  return Object.assign({}, values, {
    sourceIndex: entry.index,
    sourceId: entry.sourceId,
    signature: entry.signature,
    personalized: Boolean(course.personalized),
    weekdayText: weekdayOptions[values.weekday - 1] || "",
  });
});
});
this._baseCourses = baseCourses;
this.setData({
  targetLabel: buildTargetLabel(target),
  courses,
  sourceCourses,
  visibleSourceCourses: filterSourceCourses(sourceCourses, this.data.sourceQuery),
  sourceCount: sourceCourses.length,
  overrideCount: sourceCourses.filter((item) => item.personalized).length,
});
},
onSourceSearch(event) {
  const sourceQuery = event.detail.value || "";
  this.setData({
    sourceQuery,
    visibleSourceCourses: filterSourceCourses(this.data.sourceCourses, sourceQuery),
  });
},
showAddForm() {
  this.setData({
    formVisible: true,
    formTitle: "添加课程",
    formMode: "custom",
    sourceIndex: -1,
    form: getDefaultForm(),
  });
},
hideForm() {
  this.setData({
    formVisible: false,
  });
},
editCourse(event) {
  const id = event.currentTarget.dataset.id;
  const course = this.data.courses.find((item) => item.id === id);
  if (!course) return;
  this.setData({
    formVisible: true,
    formTitle: "编辑课程",
    formMode: "custom",
    sourceIndex: -1,
    form: courseToForm(course),
  });
}

```

```

    },
    editSourceCourse(event) {
        this.openSourceCourse(Number(event.currentTarget.dataset.index));
    },
    openSourceCourse(index) {
        const source = this._baseCourses && this._baseCourses[index];
        if (!source) return;
        const effective = courseOverrideService.applyCourseOverrides(this._baseCourses, getCurrentScheduleTarget())[index];
        const values = courseOverrideService.getSourceEditableValues(effective);
        this.setData({
            formVisible: true,
            formTitle: effective.personalized ? "调整课程" : "编辑已有课程",
            formMode: "override",
            sourceIndex: index,
            form: courseToForm(values),
        });
    },
    restoreSourceCourse(event) {
        const index = Number(event.currentTarget.dataset.index);
        const source = this._baseCourses && this._baseCourses[index];
        if (!source) return;
        wx.showModal({
            title: "恢复原课程信息",
            content: "这门课的个人调整将被移除，重新显示当前课表来源的数据。",
            confirmText: "恢复",
            success: (result) => {
                if (!result.confirm) return;
                try {
                    courseOverrideService.restoreCourseOverride(source, this._baseCourses, getCurrentScheduleTarget());
                    this.loadCourses();
                    wx.showToast({ title: "已恢复原课程", icon: "success" });
                } catch (error) {
                    wx.showToast({ title: error.message || "恢复失败", icon: "none" });
                }
            },
        });
    },
    deleteCourse(event) {
        const id = event.currentTarget.dataset.id;
        wx.showModal({
            title: "删除自定义课程",
            content: "删除后会从首页周课表和今日课程中移除。",
            confirmText: "删除",
            confirmColor: "#d94c6a",
            success: (res) => {
                if (!res.confirm) return;
                customCourseService.deleteCustomCourse(id, getCurrentScheduleTarget());
                this.loadCourses();
                wx.showToast({ title: "已删除", icon: "success" });
            },
        });
    },
    toggleCourse(event) {
        const id = event.currentTarget.dataset.id;

```

```
const enabled = event.detail.value;
customCourseService.toggleCustomCourse(id, enabled, getCurrentScheduleTarget());
this.loadCourses();
},
onInput(event) {
  const key = event.currentTarget.dataset.key;
  this.setData({
    [ `form.${key}` ]: event.detail.value,
  });
},
onPickerChange(event) {
  const key = event.currentTarget.dataset.key;
  this.setData({
    [ `form.${key}` ]: Number(event.detail.value),
  });
},
onEnabledChange(event) {
  this.setData({
    "form.enabled": event.detail.value,
  });
},
saveCourse() {
  const form = this.data.form;
  if (this.data.formMode === "override") {
    const source = this._baseCourses && this._baseCourses[this.data.sourceIndex];
    try {
      courseOverrideService.saveCourseOverride(source, {
        courseName: form.courseName,
        teacherName: form.teacherName,
        classroom: form.classroom,
        weekday: form.weekdayIndex + 1,
        startSection: form.startSectionIndex + 1,
        endSection: form.endSectionIndex + 1,
        weekText: form.weekText,
        note: form.note,
      }, this._baseCourses, getCurrentScheduleTarget());
      this.setData({ formVisible: false });
      this.loadCourses();
      wx.showToast({ title: "个人调整已保存", icon: "success" });
    } catch (error) {
      wx.showToast({ title: error.message || "保存失败", icon: "none" });
    }
    return;
  }
  const payload = {
    id: form.id,
    courseName: form.courseName,
    teacherName: form.teacherName,
    classroom: form.classroom,
    weekday: form.weekdayIndex + 1,
    weekDay: form.weekdayIndex + 1,
    startSection: form.startSectionIndex + 1,
    endSection: form.endSectionIndex + 1,
    weekText: form.weekText,
    color: form.color,
    note: form.note,
```

```

        enabled: form.enabled,
    };
    try {
        customCourseService.upsertCustomCourse(payload, getCurrentScheduleTarget());
        this.setData({
            formVisible: false,
        });
        this.loadCourses();
        wx.showToast({ title: "已保存", icon: "success" });
    } catch (error) {
        wx.showToast({
            title: error.message || "保存失败",
            icon: "none",
        });
    }
},
importXls() {
    wx.navigateTo({
        url: "/pages/personal-sync/personal-sync?tab=xls",
    });
},
exportJson() {
    const text = JSON.stringify(this.data.courses, null, 2);
    wx.setClipboardData({
        data: text,
        success: () => wx.showToast({ title: "已复制 JSON", icon: "success" }),
    });
},
importJson() {
    wx.getClipboardData({
        success: (res) => {
            try {
                const data = JSON.parse(res.data || "[]");
                if (!Array.isArray(data)) {
                    throw new Error("JSON 必须是数组");
                }
                data.forEach((item) => customCourseService.upsertCustomCourse(item, getCurrentScheduleTarget()));
            } catch (error) {
                wx.showToast({ title: "剪贴板 JSON 无效", icon: "none" });
            }
        },
    });
},
showImportPlaceholder(event) {
    const label = event.currentTarget.dataset.label || "导入";
    wx.showToast({
        title: `${label} 即将支持`,
        icon: "none",
    });
},
toggleImportTools() {
    this.setData({
        importToolsCollapsed: !this.data.importToolsCollapsed
    });
}

```

```
    });
  },
  selectColor(event) {
    const color = event.currentTarget.dataset.color;
    this.setData({
      "form.color": color
    });
  },
  noop() {},
});
// File: miniprogram/pages/empty-room/empty-room.js
const BRAND = require(".././config/brand");
const appConfigService = require(".././services/appConfigService");
const emptyRoomService = require(".././services/emptyRoomService");
const platformDataService = require(".././services/platformDataService");
const releasePackService = require(".././services/releasePackService");
const teachingCalendarService = require(".././services/teachingCalendarService");
const platformUtils = require(".././utils/platform");
const {
  addLocalDays,
  formatDate,
  getTodayTeachingInfo,
  getWeekdayLabel,
  parseLocalDate,
} = require(".././utils/week");
const DEFAULT_SEMESTER_ID = "";
const SECTION_PRESETS = [
  { key: "current", label: "当前节" },
  { key: "next", label: "下一节" },
  { key: "morning", label: "上午" },
  { key: "afternoon", label: "下午" },
  { key: "evening", label: "晚上" },
  { key: "custom", label: "自定义" },
];
const QUICK_FILTERS = [
  { key: "now", label: "现在可用" },
  { key: "morning", label: "上午" },
  { key: "afternoon", label: "下午" },
  { key: "evening", label: "晚上" },
];
const ROOM_PAGE_SIZE = 30;
const SECTION_CHIPS = Array.from({ length: 14 }, (_, index) => ({
  value: index + 1,
  label: String(index + 1),
}));
const MIN_FREE_OPTIONS = ["1", "2", "3", "4"];
function safeDecodeURIComponent(value) {
  const text = String(value || "");
  try {
    return decodeURIComponent(text);
  } catch (error) {
    return text;
  }
}
function buildDateOptions(currentDate) {
  const base = parseLocalDate(currentDate) || new Date();
```

```

const today = new Date();
const options = [
  { key: "today", label: "今天", date: formatDate(today) },
  { key: "tomorrow", label: "明天", date: formatDate(addLocalDays(today, 1)) },
];
const monday = addLocalDays(base, -((base.getDay() + 6) % 7));
for (let index = 0; index < 7; index += 1) {
  const date = addLocalDays(monday, index);
  options.push({
    key: `week-${index + 1}`,
    label: getWeekdayLabel(index + 1),
    date: formatDate(date),
  });
}
const seen = new Set();
return options.filter((item) => {
  const key = item.label + item.date;
  if (seen.has(key)) return false;
  seen.add(key);
  return true;
});
}
function parseSectionValues(value) {
  const text = String(value || "");
  const rangeMatch = text.match(/(\d+)\s*[-~至到]\s*(\d+)/);
  if (rangeMatch) {
    const start = Math.min(Number(rangeMatch[1]), Number(rangeMatch[2]));
    const end = Math.max(Number(rangeMatch[1]), Number(rangeMatch[2]));
    return Array.from({ length: end - start + 1 }, (_, index) => start + index);
  }
  return Array.from(new Set((text.match(/\d{1,2}/g) || []).map(Number).filter((num) => num
    >= 1 && num <= 14)))
    .sort((left, right) => left - right);
}
function sectionValuesToText(values) {
  const list = Array.from(new Set(values || [])).map(Number).filter((num) => num >= 1 && num <= 14).sort((left, right) => left - right);
  if (!list.length) return "";
  let contiguous = true;
  for (let index = 1; index < list.length; index += 1) {
    if (list[index] !== list[index - 1] + 1) {
      contiguous = false;
      break;
    }
  }
  return contiguous && list.length > 1 ? `${list[0]}-${list[list.length - 1]}` : list.join(",");
}
function decorateSectionChips(values) {
  const selected = new Set((values || []).map(Number));
  return SECTION_CHIPS.map((item) => Object.assign({}, item, {
    selected: selected.has(item.value),
  }));
}
function groupRoomsByBuilding(rooms, limit) {
  const max = Math.max(0, Number(limit || ROOM_PAGE_SIZE) || ROOM_PAGE_SIZE);

```

```

const visible = (rooms || []).slice(0, max);
const groups = [];
const groupMap = {};
visible.forEach((room, sourceIndex) => {
  const building = room.building || "其他";
  if (!groupMap[building]) {
    groupMap[building] = {
      building,
      rooms: [],
      total: 0,
    };
    groups.push(groupMap[building]);
  }
  groupMap[building].rooms.push(Object.assign({}, room, {
    sourceIndex,
  }));
});
(rooms || []).forEach((room) => {
  const building = room.building || "其他";
  if (groupMap[building]) groupMap[building].total += 1;
});
return groups;
}

function resolveRoomFromEvent(event, rooms, selectedRoom) {
  const dataset = event && event.currentTarget && event.currentTarget.dataset || {};
  if (dataset.room && typeof dataset.room === "object") return dataset.room;
  const rawIndex = dataset.sourceIndex !== undefined ? dataset.sourceIndex : dataset.index
;
  if (rawIndex !== undefined) {
    const index = Number(rawIndex);
    if (Number.isFinite(index) && rooms && rooms[index]) return rooms[index];
  }
  return selectedRoom || null;
}

function normalizeActiveSnapshot(config) {
  const data = config && config.data ? config.data : config;
  const version = data && data.dataVersion ? data.dataVersion : {};
  const releaseVersion = data && (data.releaseVersion || data.activeReleaseVersion || version.releaseVersion);
  if (!releaseVersion) return null;
  const updatedAt = data.dataUpdatedAt ||
    data.publishedAt ||
    version.classScheduleUpdatedAt ||
    version.classroomScheduleUpdatedAt ||
    data.updatedAt ||
    "";
  return {
    term: data.term || data.currentSemester || DEFAULT_SEMESTER_ID,
    releaseVersion,
    updatedAt,
    cacheEpoch: data.cacheEpoch || updatedAt || releaseVersion,
    counts: data.counts || {},
  };
}

function getDefaultSelectedTerm() {
  const calendar = teachingCalendarService.getImmediateActiveCalendar();

```

```
if (calendar.termConfig && calendar.termConfig.term) return calendar.termConfig.term;
const config = appConfigService.getGlobalConfig ? appConfigService.getGlobalConfig() : {
};
return config.currentSemester ||
    config.termConfig && config.termConfig.term ||
    config.availableTerms && config.availableTerms[0] && config.availableTerms[0].term ||
    DEFAULT_SEMESTER_ID;
}
Page({
  data: {
    brand: BRAND,
    activeSnapshot: null,
    date: "",
    dateText: "",
    dateOptions: [],
    datePrimaryOptions: [],
    selectedDateKey: "today",
    week: 1,
    weekday: 1,
    weekdayText: "周一",
    sectionPresets: SECTION_PRESETS,
    quickFilters: QUICK_FILTERS,
    activeQuickFilter: "now",
    sectionChips: decorateSectionChips([]),
    selectedSectionValues: [],
    selectedSectionPresetIndex: 0,
    sections: "1-1",
    customSections: "3-4",
    buildingOptions: emptyRoomService.DEFAULT_BUILDINGS,
    selectedBuildingIndex: 0,
    minFreeOptions: MIN_FREE_OPTIONS,
    selectedMinFreeIndex: 1,
    commonOnly: false,
    favoriteBuildingsOnly: false,
    excludeUnknown: true,
    showUnknownBuildings: false,
    loading: false,
    dataState: "idle",
    hasSearched: false,
    restoreHint: "",
    rooms: [],
    visibleRoomLimit: ROOM_PAGE_SIZE,
    visibleRoomGroups: [],
    summaryText: "",
    updatedAtText: "",
    buildingSummaryText: "全部教学楼",
    filterSheetVisible: false,
    termPhase: "unknown",
    isInTerm: false,
    termStatusText: "",
    isTeachingDay: true,
    teachingEventNote: "",
    selectedRoom: null,
    detailVisible: false,
    favoriteBuildings: [],
    favoriteRooms: [],
```

```

        selectedBuildingFavorite: false,
    },
    onLoad(options) {
        this.sharedOptions = options || {};
        this.favoriteState = emptyRoomService.readEmptyRoomFavorites();
        this.setData({
            favoriteBuildings: this.favoriteState.buildings,
            favoriteRooms: this.favoriteState.rooms,
        });
        this.initDefaults(options || {});
        this.prepareInitialData();
    },
    initDefaults(options) {
        const now = new Date();
        const calendar = teachingCalendarService.getImmediateActiveCalendar();
        const termConfig = calendar.termConfig || {};
        const weeks = calendar.weeks || [];
        const todayInfo = getTodayTeachingInfo(now, weeks, termConfig);
        const date = safeDecodeURIComponent(options.date) || todayInfo.date || formatDate(now)
    };

    const dateInfo = getTodayTeachingInfo(date, weeks, termConfig);
    const sectionParam = safeDecodeURIComponent(options.section || options.sections);
    const buildingParam = safeDecodeURIComponent(options.building);
    const selectedSectionPresetIndex = sectionParam
        ? SECTION_PRESETS.findIndex((item) => item.key === "custom")
        : SECTION_PRESETS.findIndex((item) => item.key === "current");
    const sections = sectionParam || emptyRoomService.getPresetSectionValue("current", now
);

    const buildingOptions = emptyRoomService.buildBuildingOptions(
        buildingParam ? [buildingParam] : [],
        (this.favoriteState && this.favoriteState.buildings) || []
    );
    const selectedBuildingIndex = buildingParam ? Math.max(0, buildingOptions.indexOf(buildingParam)) : 0;
    const selectedBuilding = buildingOptions[selectedBuildingIndex] || "全部";
    const dateOptions = buildDateOptions(date);
    this.setData({
        date,
        dateText: date,
        dateOptions,
        datePrimaryOptions: dateOptions.slice(0, 2),
        selectedDateKey: formatDate(new Date()) === date ? "today" : "",
        week: Number(options.week || dateInfo.weekNo || 1),
        weekday: Number(options.weekday || dateInfo.weekday),
        weekdayText: getWeekdayLabel(Number(options.weekday || dateInfo.weekday)),
        termPhase: dateInfo.termPhase || "unknown",
        isInTerm: Boolean(dateInfo.isInTerm),
        termStatusText: dateInfo.isInTerm ? "" : this.getTermPhaseText(dateInfo.termPhase),
        isTeachingDay: dateInfo.isTeachingDay !== false,
        teachingEventNote: dateInfo.teachingEventNote || "",
        selectedSectionPresetIndex: selectedSectionPresetIndex >= 0 ? selectedSectionPresetIndex : 0,
        sections,
        selectedSectionValues: parseSectionValues(sections),
        sectionChips: decorateSectionChips(parseSectionValues(sections)),
        customSections: sectionParam || "3-4",
    });

```

```

        buildingOptions,
        selectedBuildingIndex,
        selectedBuildingFavorite: emptyRoomService.isFavoriteBuilding(selectedBuilding, this
.favoriteState),
        buildingSummaryText: this.buildingSummaryText(buildingOptions, selectedBuildingIndex
, false),
    });
},
prepareInitialData() {
    this.setData({
        loading: false,
        dataState: "idle",
        restoreHint: "",
    });
    this.resolveActiveSnapshot()
        .then((snapshot) => {
            if (!snapshot || !snapshot.releaseVersion) {
                const error = new Error("NO_ACTIVE_RELEASE");
                error.code = "NO_ACTIVE_RELEASE";
                throw error;
            }
            this.setData({
                activeSnapshot: snapshot,
                updatedAtText: snapshot.updatedAt ? `数据更新于 ${appConfigService.formatConfigTime(snapshot.updatedAt)}` : "等待数据更新时间",
            });
            return this.loadEmptyRoomIndex({ forceNetwork: false });
        })
        .then((index) => {
            const buildingOptions = emptyRoomService.buildBuildingOptions(
                index.buildings || [],
                (this.favoriteState && this.favoriteState.buildings) || []
            );
            const currentBuilding = this.data.buildingOptions[this.data.selectedBuildingIndex]
            || "全部";
            const selectedBuildingIndex = Math.max(0, buildingOptions.indexOf(currentBuilding)
);
            this.setData({
                buildingOptions,
                selectedBuildingIndex,
                buildingSummaryText: this.buildingSummaryText(buildingOptions, selectedBuildingIndex, false),
            });
        })
        .catch((error) => {
            const state = error && error.code === "NO_ACTIVE_RELEASE" ? "noRelease" : "idle";
            this.setData({
                dataState: state,
                updatedAtText: state === "noRelease" ? "暂未发布课表数据" : "数据更新时间暂不可用",
            });
        });
},
buildingSummaryText(buildingOptions, selectedBuildingIndex, favoritesOnly) {
    const list = Array.isArray(buildingOptions) ? buildingOptions : [];
    const selected = list[selectedBuildingIndex] || "全部";

```

```

    if (favoritesOnly) return "收藏教学楼";
    if (selected && selected !== "全部") return selected;
    const count = Math.max(0, list.filter((item) => item && item !== "全部" && item !== "未知" && item !== "其他/未识别").length);
    return count ? `全部教学楼 · ${count} 栋` : "全部教学楼";
  },
  loadAndSearch(options = {}) {
    const hasRooms = Array.isArray(this.data.rooms) && this.data.rooms.length > 0;
    this.setData({
      loading: true,
      dataState: hasRooms ? this.data.dataState : "loading",
      hasSearched: true,
      restoreHint: "",
    });
    this.resolveActiveSnapshot()
      .then((snapshot) => {
        if (!snapshot || !snapshot.releaseVersion) {
          const error = new Error("NO_ACTIVE_RELEASE");
          error.code = "NO_ACTIVE_RELEASE";
          throw error;
        }
        this.setData({
          activeSnapshot: snapshot,
          updatedAtText: snapshot.updatedAt ? `数据更新于 ${appConfigService.formatConfigTime(snapshot.updatedAt)}` : "",
        });
        return this.loadEmptyRoomIndex({ forceNetwork: options.forceNetwork });
      })
      .then(() => this.applyLocalSearch({ reason: options.reason || "loadAndSearch" }))
      .catch((error) => {
        const state = error && error.code === "NO_ACTIVE_RELEASE" ? "noRelease" : "networkError";
        this.setData({
          loading: false,
          dataState: state,
          restoreHint: hasRooms ? "网络连接慢, 已保留当前结果" : "",
          visibleRoomGroups: hasRooms ? this.data.visibleRoomGroups : [],
          summaryText: hasRooms ? this.data.summaryText : (state === "noRelease" ? "暂未发布课表数据" : "空教室数据加载失败"),
        });
      });
  },
  getTermPhaseText(termPhase) {
    if (termPhase === "before-term") return "尚未开学";
    if (termPhase === "after-term") return "本学期已结束";
    if (termPhase === "unknown") return "当前不在教学周内";
    return "";
  },
  loadEmptyRoomIndex(options = {}) {
    const snapshot = this.data.activeSnapshot || {};
    const indexKey = `${snapshot.term || DEFAULT_SEMESTER_ID}:${snapshot.releaseVersion || ""}`;
    if (this.emptyRoomIndex && this.emptyRoomIndexKey === indexKey && !options.forceNetwork) {
      return Promise.resolve(this.emptyRoomIndex);
    }
  }
}

```

```

return emptyRoomService.loadEmptyRoomIndex({
  term: snapshot.term || DEFAULT_SEMESTER_ID,
  releaseVersion: snapshot.releaseVersion || "",
}, {
  forceNetwork: Boolean(options.forceNetwork),
}).then((index) => {
  this.emptyRoomIndex = index;
  this.emptyRoomIndexKey = indexKey;
  return index;
});
},
resolveActiveSnapshot() {
  const options = this.sharedOptions || {};
  const optionReleaseVersion = safeDecodeURIComponent(options.releaseVersion || options.
version);
  if (optionReleaseVersion) {
    return Promise.resolve({
      term: safeDecodeURIComponent(options.term || options.semester) || DEFAULT_SEMESTER
_ID,
      releaseVersion: optionReleaseVersion,
      updatedAt: "",
      cacheEpoch: optionReleaseVersion,
      counts: {},
    });
  }
  const platformSnapshot = platformDataService.getCachedPlatformSnapshot();
  if (platformSnapshot && platformSnapshot.releaseVersion) {
    return Promise.resolve(platformSnapshot);
  }
  const selectedTerm = safeDecodeURIComponent(options.term || options.semester) || getDe
faultSelectedTerm();
  const localActive = releasePackService.getLocalActiveRelease(selectedTerm);
  if (localActive && localActive.releaseVersion) {
    releasePackService.switchReleaseSafely({ term: localActive.term, dedupe: true })
      .catch((error) => console.warn("[empty-room] background release refresh failed", {
        code: error && (error.code || error.reasonCode),
      }));
  }
  return Promise.resolve({
    term: localActive.term,
    releaseVersion: localActive.releaseVersion,
    updatedAt: localActive.manifest && localActive.manifest.updatedAt || "",
    cacheEpoch: localActive.cacheEpoch,
    forceRefreshToken: localActive.forceRefreshToken,
    counts: localActive.manifest && localActive.manifest.counts || {},
  });
}
return releasePackService.switchReleaseSafely({ term: selectedTerm, dedupe: true })
  .then((result) => normalizeActiveSnapshot(result && result.manifest))
  .catch(() => appConfigService.loadAppConfig()
    .then((config) => normalizeActiveSnapshot(config))
    .catch(() => platformDataService.loadPrefetchData({ network: false }).then((data)
=> platformDataService.extractActiveSnapshot(data))));
},
getQueryParams() {
  const snapshot = this.data.activeSnapshot || {};
  const building = this.data.buildingOptions[this.data.selectedBuildingIndex] || "全部";

```

```

return {
  term: snapshot.term || DEFAULT_SEMESTER_ID,
  releaseVersion: snapshot.releaseVersion || "",
  date: this.data.date,
  week: this.data.week,
  weekday: this.data.weekday,
  sections: this.data.sections || this.data.customSections,
  building,
  minFreeSections: this.data.minFreeOptions[this.data.selectedMinFreeIndex] || "1",
  commonOnly: this.data.commonOnly ? "1" : "",
  excludeUnknown: this.data.excludeUnknown ? "1" : "",
};
},
applyLocalSearch(options = {}) {
  const params = this.getQueryParams();
  if (this.data.isTeachingDay === false) {
    this.setData({
      loading: false,
      dataState: "empty",
      rooms: [],
      visibleRoomGroups: [],
      summaryText: this.data.teachingEventNote || "当日按校历不执行常规教学安排",
      restoreHint: "请选择其他教学日期",
    });
    return Promise.resolve({ success: true, code: "CALENDAR_NO_CLASS", rooms: [] });
  }
  if (this.data.isInTerm === false && this.data.activeQuickFilter === "now" && !options.
allowOutOfTermNow) {
    this.setData({
      loading: false,
      dataState: "empty",
      rooms: [],
      visibleRoomGroups: [],
      summaryText: this.data.termStatusText || "当前不在教学周内",
      restoreHint: "可手动选择历史日期和节次后查询",
    });
    return Promise.resolve({
      success: false,
      code: "TERM_PHASE_OUT_OF_TERM",
      termPhase: this.data.termPhase,
      rooms: [],
    });
  }
  const index = this.emptyRoomIndex;
  if (!index) {
    return this.loadEmptyRoomIndex({ forceNetwork: Boolean(options.forceNetwork) })
      .then(() => this.applyLocalSearch(Object.assign({}, options, { forceNetwork: false
})));
  }
  try {
    const data = emptyRoomService.filterEmptyRoomIndex(index, params);
    let rooms = this.decorateRoomsWithFavorites(data.rooms || []);
    if (this.data.favoriteBuildingsOnly) {
      const favoriteBuildings = new Set((this.favoriteState && this.favoriteState.buildi
ngs) || []);
      rooms = rooms.filter((room) => favoriteBuildings.has(room.building));
    }
  }
}

```

```

    }
    const buildingOptions = emptyRoomService.buildBuildingOptions(
      index.buildings || data.buildings || [],
      (this.favoriteState && this.favoriteState.buildings) || []
    );
    const currentBuilding = params.building || "全部";
    const selectedBuildingIndex = Math.max(0, buildingOptions.indexOf(currentBuilding));
    const visibleRoomLimit = ROOM_PAGE_SIZE;
    this.setData({
      loading: false,
      dataState: rooms.length ? "success" : "empty",
      hasSearched: true,
      rooms,
      visibleRoomLimit,
      visibleRoomGroups: groupRoomsByBuilding(rooms, visibleRoomLimit),
      buildingOptions,
      selectedBuildingIndex,
      selectedBuildingFavorite: emptyRoomService.isFavoriteBuilding(buildingOptions[selectedBuildingIndex], this.favoriteState),
      restoreHint: data.fromStorage ? "已显示本地缓存，筛选在本地完成" : "",
      summaryText: `${params.date} ${getWeekdayLabel(Number(params.weekday))} 第${params.sections}节 · ${rooms.length}间可用`,
      updatedAtText: data.updatedAt ? `数据更新于 ${appConfigService.formatConfigTime(data.updatedAt)}` : this.data.updatedAtText,
      buildingSummaryText: this.buildingSummaryText(buildingOptions, selectedBuildingIndex, this.data.favoriteBuildingsOnly),
    });
    return Promise.resolve(data);
  } catch (error) {
    this.setData({
      loading: false,
      dataState: error && (error.code === "REQUEST_TIMEOUT" || error.code === "TIMEOUT") ? "timeout" : "networkError",
      restoreHint: this.data.rooms.length ? "网络连接慢，已保留当前结果" : "",
      summaryText: this.data.rooms.length ? this.data.summaryText : "空教室数据加载失败"
    },
    );
    return Promise.reject(error);
  }
},
searchRooms(options = {}) {
  if (options.forceNetwork) {
    return this.loadEmptyRoomIndex({ forceNetwork: true })
      .then(() => this.applyLocalSearch(options))
      .catch((error) => {
        const rooms = Array.isArray(this.data.rooms) ? this.data.rooms : [];
        const hasRooms = rooms.length > 0;
        this.setData({
          loading: false,
          dataState: hasRooms ? this.data.dataState || "success" : "networkError",
          restoreHint: hasRooms ? "网络连接慢，已保留当前结果" : "",
          visibleRoomGroups: hasRooms ? groupRoomsByBuilding(rooms, this.data.visibleRoomLimit || ROOM_PAGE_SIZE) : [],
          summaryText: hasRooms ? this.data.summaryText : "空教室数据加载失败",
        });
        return Promise.reject(error);
      });
  }
}

```

```

    });
  }
  return this.applyLocalSearch(options);
},
onQueryTap() {
  return this.loadAndSearch({ reason: "manual-query" });
},
openFilterSheet() {
  this.setData({ filterSheetVisible: true });
},
closeFilterSheet() {
  this.setData({ filterSheetVisible: false });
},
resetFilters() {
  this.initDefaults(this.sharedOptions || {});
},
confirmFilters() {
  this.setData({
    filterSheetVisible: false,
    buildingSummaryText: this.buildingSummaryText(this.data.buildingOptions, this.data.s
electedBuildingIndex, this.data.favoriteBuildingsOnly),
  });
},
onReachBottom() {
  const rooms = this.data.rooms || [];
  if (!rooms.length || this.data.visibleRoomLimit >= rooms.length) return;
  const visibleRoomLimit = Math.min(rooms.length, this.data.visibleRoomLimit + ROOM_PAGE
_SIZE);
  this.setData({
    visibleRoomLimit,
    visibleRoomGroups: groupRoomsByBuilding(rooms, visibleRoomLimit),
  });
},
decorateRoomsWithFavorites(rooms) {
  const favorites = this.favoriteState || emptyRoomService.readEmptyRoomFavorites();
  return (rooms || []).map((room) => Object.assign({}, room, {
    favorite: emptyRoomService.isFavoriteRoom(room.roomName, favorites),
  }));
},
refreshFavoriteState() {
  this.favoriteState = emptyRoomService.readEmptyRoomFavorites();
  const building = this.data.buildingOptions[this.data.selectedBuildingIndex] || "全部";
  const rooms = this.decorateRoomsWithFavorites(this.data.rooms);
  this.setData({
    favoriteBuildings: this.favoriteState.buildings,
    favoriteRooms: this.favoriteState.rooms,
    selectedBuildingFavorite: emptyRoomService.isFavoriteBuilding(building, this.favorit
eState),
    rooms,
    visibleRoomGroups: groupRoomsByBuilding(rooms, this.data.visibleRoomLimit),
  });
},
onDateChange(event) {
  const date = event.detail.value;
  const calendar = teachingCalendarService.getImmediateActiveCalendar();

```

```

    const info = getTodayTeachingInfo(date, calendar.weeks || [], calendar.termConfig || {
});
    this.setData({
        date,
        dateText: date,
        dateOptions: buildDateOptions(date),
        datePrimaryOptions: buildDateOptions(date).slice(0, 2),
        selectedDateKey: formatDate(new Date()) === date ? "today" : "",
        week: info.weekNo,
        weekday: info.weekday,
        weekdayText: info.weekdayLabel,
        termPhase: info.termPhase || "unknown",
        isInTerm: Boolean(info.isInTerm),
        termStatusText: info.isInTerm ? "" : this.getTermPhaseText(info.termPhase),
        isTeachingDay: info.isTeachingDay !== false,
        teachingEventNote: info.teachingEventNote || "",
        activeQuickFilter: "custom",
    });
},
onDateChipTap(event) {
    const date = event.currentTarget.dataset.date;
    const key = event.currentTarget.dataset.key || "";
    if (!date) return;
    const calendar = teachingCalendarService.getImmediateActiveCalendar();
    const info = getTodayTeachingInfo(date, calendar.weeks || [], calendar.termConfig || {
});
    this.setData({
        date,
        dateText: date,
        dateOptions: buildDateOptions(date),
        datePrimaryOptions: buildDateOptions(date).slice(0, 2),
        selectedDateKey: key,
        week: info.weekNo,
        weekday: info.weekday,
        weekdayText: info.weekdayLabel,
        termPhase: info.termPhase || "unknown",
        isInTerm: Boolean(info.isInTerm),
        termStatusText: info.isInTerm ? "" : this.getTermPhaseText(info.termPhase),
        isTeachingDay: info.isTeachingDay !== false,
        teachingEventNote: info.teachingEventNote || "",
        activeQuickFilter: key === "today" ? this.data.activeQuickFilter : "custom",
    });
},
onQuickFilterTap(event) {
    const key = event.currentTarget.dataset.key;
    const patch = {
        activeQuickFilter: key,
        favoriteBuildingsOnly: false,
    };
    if (key === "now") {
        patch.sections = emptyRoomService.getPresetSectionValue("current", new Date());
        patch.selectedSectionPresetIndex = SECTION_PRESETS.findIndex((item) => item.key ===
"current");
        patch.selectedMinFreeIndex = 0;
    } else if (key === "morning" || key === "afternoon" || key === "evening") {
        patch.sections = emptyRoomService.getPresetSectionValue(key, new Date());
    }
}

```

```

    patch.selectedSectionPresetIndex = SECTION_PRESETS.findIndex((item) => item.key ===
key);
    patch.selectedMinFreeIndex = 0;
  } else if (key === "continuous2") {
    patch.selectedMinFreeIndex = 1;
  } else if (key === "continuous4") {
    patch.selectedMinFreeIndex = 3;
  } else if (key === "favorites") {
    patch.favoriteBuildingsOnly = true;
  }
  if (patch.sections) {
    patch.selectedSectionValues = parseSectionValues(patch.sections);
    patch.sectionChips = decorateSectionChips(patch.selectedSectionValues);
    patch.customSections = patch.sections;
  }
  this.setData(patch);
},
onSectionChipTap(event) {
  const value = Number(event.currentTarget.dataset.value);
  if (!value) return;
  const current = new Set(this.data.selectedSectionValues || []);
  if (current.has(value)) {
    current.delete(value);
  } else {
    current.add(value);
  }
  const selectedSectionValues = Array.from(current).sort((left, right) => left - right);
  const sections = sectionValuesToText(selectedSectionValues);
  this.setData({
    selectedSectionValues,
    sectionChips: decorateSectionChips(selectedSectionValues),
    sections: sections || this.data.sections,
    customSections: sections || this.data.customSections,
    selectedSectionPresetIndex: SECTION_PRESETS.findIndex((item) => item.key === "custom
"),
    activeQuickFilter: "custom",
  });
},
onSectionGroupTap(event) {
  const key = event.currentTarget.dataset.key;
  const sections = emptyRoomService.getPresetSectionValue(key, new Date());
  this.setData({
    sections,
    customSections: sections,
    selectedSectionValues: parseSectionValues(sections),
    sectionChips: decorateSectionChips(parseSectionValues(sections)),
    selectedSectionPresetIndex: SECTION_PRESETS.findIndex((item) => item.key === key),
    activeQuickFilter: key,
  });
},
onBuildingChipTap(event) {
  const building = event.currentTarget.dataset.building || "全部";
  const selectedBuildingIndex = Math.max(0, this.data.buildingOptions.indexOf(building))
;
  this.setData({
    selectedBuildingIndex,

```

```
        selectedBuildingFavorite: emptyRoomService.isFavoriteBuilding(building, this.favoriteState),
        favoriteBuildingsOnly: false,
        buildingSummaryText: this.buildingSummaryText(this.data.buildingOptions, selectedBuildingIndex, false),
    });
},
toggleUnknownBuildings() {
    const showUnknownBuildings = !this.data.showUnknownBuildings;
    this.setData({
        showUnknownBuildings,
        excludeUnknown: !showUnknownBuildings,
    });
},
onWeekInput(event) {
    const week = Math.max(1, Math.min(30, Number(event.detail.value) || 1));
    this.setData({ week });
},
applyWeekInput() {
    this.setData({ filterSheetVisible: false });
},
onSectionPresetChange(event) {
    const selectedSectionPresetIndex = Number(event.detail.value);
    const preset = SECTION_PRESETS[selectedSectionPresetIndex] || SECTION_PRESETS[0];
    const sections = preset.key === "custom"
        ? this.data.customSections
```

```

    });
  }
  wx.showToast({
    title: nextEnabled ? "已收藏教室" : "已取消收藏",
    icon: "none",
  });
},
openClassroomSchedule(event) {
  const room = resolveRoomFromEvent(event, this.data.rooms, this.data.selectedRoom);
  if (!room) return;
  const snapshot = this.data.activeSnapshot || {};
  const term = snapshot.term || DEFAULT_SEMESTER_ID;
  const releaseVersion = snapshot.releaseVersion || "";
  const localActive = releasePackService.getLocalActiveRelease(term);
  const activeManifest = localActive &&
    localActive.releaseVersion === releaseVersion &&
    localActive.manifest
    ? localActive.manifest
    : null;
  wx.showLoading({ title: "正在打开课表...", mask: true });
  releasePackService.resolveClassroomDetail(room.roomName, {
    term,
    releaseVersion,
    detailId: room.detailId || room.classroomId || "",
  }, {
    manifest: activeManifest,
    timeout: 15000,
    retries: 0,
  })
  .then((resolved) => {
    wx.hideLoading();
    const schedule = resolved.schedule || resolved.detail || {};
    const detailId = resolved.detailId || resolved.resolvedId || schedule.id || room.d
etailId || room.classroomId || "";
    if (!detailId) {
      throw Object.assign(new Error("CLASSROOM_DETAIL_ID_MISSING"), { code: "CLASSROOM
_DETAIL_ID_MISSING" });
    }
    wx.navigateTo({
      url: ` /pages/schedule-view/schedule-view?type=classroom&id=${encodeURIComponent(
detailId)}&name=${encodeURIComponent(room.roomName)}&term=${encodeURIComponent(term)}&seme
ster=${encodeURIComponent(term)}&releaseVersion=${encodeURIComponent(resolved.releaseVersi
on || releaseVersion)}`,
      success: (res) => {
        if (res.eventChannel && typeof res.eventChannel.emit === "function") {
          res.eventChannel.emit("acceptDataFromOpenerPage", {
            courses: Array.isArray(schedule.courses) ? schedule.courses : [],
            schedule: Object.assign({}, schedule, {
              id: detailId,
              detailId,
              roomName: room.roomName,
              scheduleVersion: resolved.releaseVersion || releaseVersion,
              releaseVersion: resolved.releaseVersion || releaseVersion,
              semester: term,
            }),
          });
        }
      }
    );
  });
}

```

```
    }
  },
});
})
.catch((error) => {
  wx.hideLoading();
  wx.showToast({
    title: "该教室暂无课表详情，但空闲结果仍可参考",
    icon: "none",
  });
  if (platformUtils.isDeveloperEnv && platformUtils.isDeveloperEnv()) {
    console.warn("[empty-room] open classroom schedule failed", {
      roomName: room.roomName,
      releaseVersion,
      error,
    });
  }
});
},
});
// File: miniprogram/pages/index/index.js
const { courseTimes } = require("../data/courseTimes");
const { buildScheduleColumns, getCourseDataSource, getCoursesByClass } = require("../utils/course");
const { getSettings, saveSettings } = require("../utils/storage");
const { resolveAdjacentWeek, resolveWeekSwipeDirection } = require("../utils/weekSwipe");
const { getTodayCoursesData, shouldShowTodayStartupReminder } = require("../utils/todayReminder");
const appConfigService = require("../services/appConfigService");
const dailyKnowledgeCloudService = require("../services/dailyKnowledgeCloudService");
const customCourseService = require("../services/customCourseService");
const currentScheduleService = require("../services/currentScheduleService");
const teachingCalendarService = require("../services/teachingCalendarService");
const { buildWeekPickerOptions } = require("../utils/weekPicker");
const courseOverrideService = require("../services/courseOverrideService");
const BRAND = require("../config/brand");
const {
  TOTAL_WEEKS,
  addLocalDays,
  clampWeek,
  formatDate,
  formatDateLabel,
  formatWeekRange,
  getCurrentTeachingWeek,
  getTeachingPeriodText,
  getTodayTeachingInfo,
  getVisibleWeekdays,
  getWeekRangeByWeekNo,
} = require("../utils/week");
const PAGE_PADDING_RPX = 32;
const TIME_AXIS_WIDTH = 76;
const WEEKEND_DAY_WIDTH = 142;
function getContentWidthRpx() {
  return 750 - PAGE_PADDING_RPX;
}
```

```

function resolveDisplayWeek(settings, todayInfo, termConfig) {
  if (settings.manualWeekOverride) {
    return clampWeek(settings.currentWeek, termConfig);
  }
  return todayInfo.weekNo;
}

function calendarChanged(left, right) {
  if (!left || !right) return Boolean(left || right);
  return left.term !== right.term ||
    left.releaseVersion !== right.releaseVersion ||
    JSON.stringify(left.termConfig || {}) !== JSON.stringify(right.termConfig || {}) ||
    JSON.stringify((left.weeks || []).map((week) => [week.weekNo, week.startDate, week.endDate, week.note || week.notes || ""])) !==
    JSON.stringify((right.weeks || []).map((week) => [week.weekNo, week.startDate, week.endDate, week.note || week.notes || ""]));
}

function buildPersonalXlsHeader(target) {
  const metadata = (target && target.metadata) || {};
  const title = target.title || target.name || (metadata.className
    ? `${metadata.className}课表`
    : (metadata.studentName ? `${metadata.studentName}的课表` : "个人课表"));
  const term = metadata.term || target.semester || "";
  const subtitle = target.subtitle || [metadata.studentName, term, "XLS 导入"].filter(Boolean).join(" · ");
  return {
    title,
    subtitle,
    sourceText: target.sourceText || "100 网 XLS 手动导入",
  };
}

function buildPersonalApaasHeader(target) {
  const metadata = (target && target.metadata) || {};
  const title = target.title || target.name || (metadata.studentName ? `${metadata.studentName}的个人课表` : "个人课表");
  const term = metadata.term || target.semester || "";
  const subtitle = target.subtitle || [metadata.className || "班级未确认", term, "学号导入"].filter(Boolean).join(" · ");
  return {
    title,
    subtitle,
    sourceText: target.sourceText || "学校课表系统",
  };
}

function decorateUnplacedCourses(courses) {
  return (Array.isArray(courses) ? courses : []).map((course, index) => ({
    id: course.id || course.arrangementId || `unplaced-${index}`,
    courseName: course.displayCourseName || course.courseName || "未命名课程",
    reason: course.reason || course.note || course.specialNote || "时间信息需要确认",
    teacherName: course.teacherName || "",
    roomName: course.roomName || course.classroom || "未注明",
    weekText: course.weekText || "周次待确认",
    sectionText: course.sectionText || "节次待确认",
  }));
}

Page({
  data: {

```

```
brand: BRAND,
logoPath: "/assets/logo/favicon.png",
showLogo: true,
appName: BRAND.appName,
className: "未选择课表",
scheduleSubtitle: "",
semester: "",
dataSourceText: "课程数据 · 本地缓存",
lastSyncText: "",
syncActionText: "同步课表",
currentWeek: 12,
teachingPeriodText: "教学周待同步",
termPhase: "unknown",
totalWeeks: TOTAL_WEEKS,
weekRangeText: "",
weekScopeText: "周一至周五",
todayText: "",
weekSwitcherLabel: "",
weekPickerOpen: false,
weekOptions: [],
sections: courseTimes,
sectionHeight: 90,
scheduleHeight: courseTimes.length * 90,
gridWidth: 718,
dayTrackWidth: 642,
dayColumnWidth: 128,
weekdays: [],
dayColumns: [],
hideInactiveCourses: true,
showWeekend: true,
weekendShowMode: "overview",
selectedCourse: null,
detailVisible: false,
unplacedCourses: [],
unplacedCourseCount: 0,
showUnplacedCourses: false,
// 新增状态
showInitModal: false,
showTodayReminder: false,
todayReminderData: null,
hasBoundTarget: false,
showDisclaimerPopup: false,
showMoreMenu: false,
appConfig: { notices: [], banners: [], news: [], appConfig: {} },
dataVersionText: "",
homeNotice: null,
dailyKnowledge: null,
tickerNotice: null,
modalNotice: null,
selectedNotice: null,
showNoticeDetail: false,
showAppNoticeModal: false,
},
onLoad(options) {
  this._weekSwipeState = null;
  this._suppressCourseTapUntil = 0;
```

```

    if (options && options.shareScheduleId) {
      wx.navigateTo({
        url: `/pages/schedule-view/schedule-view?type=class&name=${encodeURIComponent(options.shareScheduleName || "")}&shareScheduleId=${options.shareScheduleId}`
      });
      return;
    }
  },
  onShow() {
    const { isScheduleInitialized, getCurrentScheduleTarget } = require("../utils/storage");
    const initialized = isScheduleInitialized();
    const target = getCurrentScheduleTarget();
    if (!initialized || !target) {
      this.setData({
        showInitModal: true,
        hasBoundTarget: false,
        className: "未选择课表",
      });
      this.loadSchedule();
      this.loadPageConfig();
    } else {
      this.setData({
        showInitModal: false,
        hasBoundTarget: true,
      });
      this.loadSchedule();
      this.loadPageConfig();
      this.checkTodayReminder();
      this.refreshCurrentTargetSilently();
    }
  },
  refreshCurrentTargetSilently() {
    currentScheduleService.ensureCurrentScheduleFresh({
      silent: true,
      notify: true,
    }).then((result) => {
      if (result && result.status === "UPDATED") {
        this.loadSchedule();
        if (result.shouldNotify) {
          wx.showToast({
            title: "课表已更新至最新数据",
            icon: "none",
            duration: 1200,
          });
        }
      }
    }).catch(() => {});
  },
  loadPageConfig() {
    return appConfigService.loadAppConfig()
      .then((config) => {
        const normalizedConfig = appConfigService.normalizeConfig
          ? appConfigService.normalizeConfig(config)
          : Object.assign({ notices: [], banners: [], news: [], appConfig: {} }, config || {});
      });
  }
}

```

```

        const homeNotice = appConfigService.getPrimaryNotice(normalizedConfig, "home", ["b
anner", "card"]) || null;
        const tickerNotice = appConfigService.getPrimaryNotice(normalizedConfig, "home", [
"ticker"]) || null;
        const modalNotice = appConfigService.getPrimaryNotice(normalizedConfig, "home", ["
modal"]) || null;
        const latestUpdatedAt = appConfigService.getLatestDataUpdatedAt(normalizedConfig);
        const dataVersionText = latestUpdatedAt
            ? `数据更新于 ${appConfigService.formatConfigTime(latestUpdatedAt)}`
            : "";
        const app = getApp();
        const modalKey = modalNotice ? `${modalNotice.id}:${modalNotice.version}` : "";
        const shouldShowModal = modalNotice &&
            appConfigService.shouldShowNotice(modalNotice) &&
            !(app.globalData.shownModalNoticeIds || {})[modalKey];
        if (shouldShowModal) {
            app.globalData.shownModalNoticeIds = Object.assign({}, app.globalData.shownModal
NoticeIds, {
                [modalKey]: true,
            });
        }
        const serverDailyKnowledge = normalizedConfig.dailyKnowledge || null;
        this.setData({
            appConfig: normalizedConfig,
            dataVersionText,
            homeNotice,
            dailyKnowledge: serverDailyKnowledge,
            tickerNotice,
            modalNotice,
            showAppNoticeModal: Boolean(shouldShowModal),
        });
        dailyKnowledgeCloudService.loadDailyKnowledge({ fallback: serverDailyKnowledge })
            .then((dailyKnowledge) => {
                if (dailyKnowledge) this.setData({ dailyKnowledge });
            })
            .catch(() => {});
    })
    .catch((err) => {
        console.warn("首页公告配置加载失败", err);
        this.setData({
            appConfig: appConfigService.normalizeConfig
                ? appConfigService.normalizeConfig(this.data.appConfig)
                : Object.assign({ notices: [], banners: [], news: [], appConfig: {} }, this.da
ta.appConfig || {}),
        });
    });
},
loadSchedule() {
    const calendar = teachingCalendarService.getImmediateActiveCalendar();
    this.renderScheduleWithCalendar(calendar);
    teachingCalendarService.loadActiveTeachingCalendar()
        .then((latest) => {
            if (calendarChanged(calendar, latest)) {
                this.renderScheduleWithCalendar(latest);
            }
        })
}

```

```

        .catch(() => {});
    },
    renderScheduleWithCalendar(calendar) {
        const settings = getSettings();
        const { getCurrentScheduleTarget } = require("../utils/storage");
        const target = getCurrentScheduleTarget();
        const hasBoundTarget = !!target;
        const termConfig = calendar.termConfig || {};
        const calendarWeeks = calendar.weeks || [];
        const now = new Date();
        const todayInfo = getTodayTeachingInfo(now, calendarWeeks, termConfig);
        const currentWeek = resolveDisplayWeek(settings, todayInfo, termConfig);
        const weekInfo = getWeekRangeByWeekNo(currentWeek, calendarWeeks, termConfig);
        const showWeekend = settings.showWeekend || false;
        const weekendShowMode = settings.weekendShowMode || "overview";
        const baseWeekdays = getVisibleWeekdays(showWeekend, now);
        const weekdays = baseWeekdays.map((day, index) => {
            const date = addLocalDays(weekInfo.startDate, index);
            const dateInfo = getTodayTeachingInfo(date, calendarWeeks, termConfig);
            return Object.assign({}, day, {
                date: formatDate(date),
                dateLabel: formatDateLabel(date),
                isToday: currentWeek === todayInfo.rawWeekNo && day.weekday === todayInfo.physical
            });
        });
        const teachingDay = dateInfo.isTeachingDay,
            scheduleWeek = dateInfo.weekNo,
            scheduleWeekday = dateInfo.weekday,
            teachingEventType = dateInfo.teachingEventType,
            teachingEventNote = dateInfo.teachingEventNote,
            isToday = currentWeek === todayInfo.rawWeekNo && day.weekday === todayInfo.physical;
        const courses = getCoursesByClass(settings.className);
        const dataSource = getCourseDataSource();
        const dayColumns = buildScheduleColumns(courses, weekdays, currentWeek, {
            sectionHeight: 90,
            hideInactiveCourses: settings.hideInactiveCourses,
            normalized: true,
            targetType: target && target.type || "class",
            targetId: target && (target.detailId || target.id || target.classId) || settings.classId || "",
            targetName: target && (target.name || target.className) || settings.className || "",
            semester: target && (target.term || target.semester) || termConfig.term || "",
            releaseVersion: target && (target.scheduleVersion || target.releaseVersion) || calendar.releaseVersion || "",
        });
        const contentWidth = getContentWidthRpx();
        let dayColumnWidth = 128;
        let scrollX = false;
        if (showWeekend) {
            if (weekendShowMode === "detail") {
                dayColumnWidth = WEEKEND_DAY_WIDTH; // 142
                scrollX = true;
            } else {
                // 七天概览模式，一屏显示周一至周日，无横滚
                dayColumnWidth = Math.floor((contentWidth - TIME_AXIS_WIDTH) / 7); // (750 - 32 - 76) / 7 = 91
            }
        }
    }
}

```

```

        scrollX = false;
    }
} else {
    // 五天详细，一屏无横滚
    dayColumnWidth = Math.floor((contentWidth - TIME_AXIS_WIDTH) / 5); // 128
    scrollX = false;
}
const dayTrackWidth = dayColumnWidth * weekdays.length;
const gridWidth = TIME_AXIS_WIDTH + dayTrackWidth;
const weekRangeText = formatWeekRange(weekInfo.startDate, weekInfo.endDate);
const weekSwitcherLabel = weekRangeText ? `${weekRangeText} · 第${currentWeek}周` : `
日期待同步 · 第${currentWeek}周`;
let displayClassName = settings.className || "未选择课表";
let scheduleSubtitle = "";
let lastSyncText = "";
let syncActionText = "同步课表";
let sourceText = dataSource.text;
if (target) {
    if (target.type === "personal-xls" || target.type === "personal-apaas") {
        const header = target.type === "personal-apaas"
            ? buildPersonalApaasHeader(target)
            : buildPersonalXlsHeader(target);
        displayClassName = header.title;
        scheduleSubtitle = header.subtitle;
        sourceText = header.sourceText;
        syncActionText = "更新导入";
    } else {
        displayClassName = target.type === "teacher"
            ? `${target.name} 老师`
            : (target.type === "classroom" ? `${target.name} 教室` : target.name);
    }
    lastSyncText = target.updateTime || "";
}
const unplacedCourses = decorateUnplacedCourses(target && target.unplacedCourses);
this.setData({
    className: displayClassName,
    scheduleSubtitle,
    semester: calendar.semesterText || termConfig.semesterText || termConfig.term || "",
    dataSourceText: sourceText,
    lastSyncText,
    syncActionText,
    currentWeek,
    teachingPeriodText: getTeachingPeriodText(todayInfo, currentWeek),
    termPhase: todayInfo.termPhase || "unknown",
    totalWeeks: termConfig.totalWeeks || TOTAL_WEEKS,
    weekRangeText,
    weekScopeText: showWeekend ? "周一至周日" : "周一至周五",
    todayText: `${todayInfo.dateLabel} ${todayInfo.weekdayLabel}`,
    weekSwitcherLabel,
    weekOptions: buildWeekPickerOptions(calendar),
    gridWidth,
    dayTrackWidth,
    dayColumnWidth,
    weekdays,
    dayColumns,
    hideInactiveCourses: settings.hideInactiveCourses,

```

```

        showWeekend: showWeekend,
        weekendShowMode: weekendShowMode,
        scrollX: scrollX,
        hasBoundTarget,
        unplacedCourses,
        unplacedCourseCount: unplacedCourses.length,
    });
},
onWeekChange(event) {
    const detail = event && event.detail || {};
    const type = detail.type;
    if (type !== "prev" && type !== "next" && type !== "current" && type !== "select") return;
    const calendar = teachingCalendarService.getImmediateActiveCalendar();
    const termConfig = calendar.termConfig || {};
    const nextWeek = type === "current"
        ? getCurrentTeachingWeek(new Date(), calendar.weeks || [], termConfig)
        : clampWeek(detail.week, termConfig);
    if (type !== "current" && nextWeek === this.data.currentWeek) return;
    saveSettings({
        currentWeek: nextWeek,
        manualWeekOverride: type !== "current",
    });
    this.loadSchedule();
},
onWeekPickerModalChange(event) {
    this.setData({ weekPickerOpen: Boolean(event.detail && event.detail.visible) });
},
canHandleScheduleSwipe() {
    return !(this.data.showWeekend && this.data.weekendShowMode === "detail") &&
        !this.data.weekPickerOpen &&
        !this.data.detailVisible &&
        !this.data.showUnplacedCourses &&
        !this.data.showTodayReminder &&
        !this.data.showDisclaimerPopup &&
        !this.data.showMoreMenu &&
        !this.data.showNoticeDetail &&
        !this.data.showAppNoticeModal;
},
onScheduleTouchStart(event) {
    const touches = event && event.touches || [];
    if (!this.canHandleScheduleSwipe() || touches.length !== 1) {
        this._weekSwipeState = null;
        return;
    }
    const touch = touches[0];
    this._weekSwipeState = {
        start: { clientX: touch.clientX, clientY: touch.clientY },
        last: { clientX: touch.clientX, clientY: touch.clientY },
    };
},
onScheduleTouchMove(event) {
    const touches = event && event.touches || [];
    if (!this._weekSwipeState || touches.length !== 1) return;
    const touch = touches[0];
    this._weekSwipeState.last = { clientX: touch.clientX, clientY: touch.clientY };
}

```

```

},
onScheduleTouchEnd(event) {
  const swipeState = this._weekSwipeState;
  this._weekSwipeState = null;
  if (!swipeState || !this.canHandleScheduleSwipe()) return;
  const changedTouches = event && event.changedTouches || [];
  const end = changedTouches.length === 1 ? changedTouches[0] : swipeState.last;
  const direction = resolveWeekSwipeDirection(swipeState.start, end);
  if (!direction) return;
  // A committed horizontal gesture must not also open the course card that
  // happened to be under the finger when touchend fired.
  this._suppressCourseTapUntil = Date.now() + 240;
  const adjacent = resolveAdjacentWeek(this.data.currentWeek, this.data.totalWeeks, direction);
  if (!adjacent.changed) return;
  this.onWeekChange({
    detail: {
      type: direction,
      week: adjacent.week,
      source: "swipe",
    },
  });
},
onScheduleTouchCancel() {
  this._weekSwipeState = null;
},
onCourseTap(event) {
  if (Date.now() < Number(this._suppressCourseTapUntil || 0)) return;
  this.setData({
    selectedCourse: event.detail.course,
    detailVisible: true,
  });
},
closeCourseDetail() {
  this.setData({
    detailVisible: false,
    selectedCourse: null,
  });
},
showUnplacedCourses() {
  if (!this.data.unplacedCourseCount) return;
  this.setData({ showUnplacedCourses: true });
},
closeUnplacedCourses() {
  this.setData({ showUnplacedCourses: false });
},
onCopyCourseToCustom(event) {
  try {
    customCourseService.saveCustomCourseDraft(event.detail.course || this.data.selectedCourse);
    this.closeCourseDetail();
    wx.navigateTo({
      url: "/pages/custom-courses/custom-courses",
    });
  } catch (error) {
    wx.showToast({

```

```
        title: "课程信息不完整",
        icon: "none",
    });
}
},
onEditExistingCourse(event) {
    try {
        courseOverrideService.saveEditDraft(event.detail.course || this.data.selectedCourse)
;
        this.closeCourseDetail();
        wx.navigateTo({ url: "/pages/custom-courses/custom-courses" });
    } catch (error) {
        wx.showToast({ title: "请从个性化页面选择课程", icon: "none" });
    }
},
onLogoError() {
    this.setData({
        showLogo: false,
    });
},
goSchool() {
    wx.switchTab({
        url: "/pages/school/school",
    });
},
goLogin() {
    wx.navigateTo({
        url: "/pages/personal-sync/personal-sync"
    });
},
goTimetable() {
    wx.navigateTo({
        url: "/pages/timetable/timetable",
    });
},
goCustomCourses() {
    wx.navigateTo({
        url: "/pages/custom-courses/custom-courses",
    });
},
goAiAssistant() {
    this.hideMoreMenu();
    wx.navigateTo({
        url: "/packageXiaofu/pages/ai-assistant/ai-assistant",
    });
},
// 引导弹窗方法
goToSelectClass() {
    this.setData({ showInitModal: false });
    wx.switchTab({
        url: "/pages/school/school"
    });
},
skipSelect() {
    const { clearCurrentScheduleTarget } = require("../utils/storage");
    clearCurrentScheduleTarget();
}
```

```
wx.setStorageSync("hasInitializedSchedule", true);
this.setData({
  showInitModal: false,
  hasBoundTarget: false,
  className: "请选择课表"
});
this.loadSchedule();
},
goToSyncLogin() {
  wx.navigateTo({
    url: "/pages/personal-sync/personal-sync"
  });
},
// 今日课程自动弹窗提醒
checkTodayReminder() {
  const settings = getSettings();
  if (settings.enableTodayStartupReminder === false) {
    return;
  }
  const app = getApp();
  if (app.globalData.hasShownTodayReminderThisSession) {
    return;
  }
  const now = new Date();
  const pad = (value) => String(value).padStart(2, "0");
  const todayDateText = `${now.getFullYear()}-${pad(now.getMonth() + 1)}-${pad(now.getDate())}`;
  const lastDate = wx.getStorageSync("lastTodayReminderDate");
  if (lastDate === todayDateText) {
    return;
  }
  const todayData = getTodayCoursesData();
  if (!shouldShowTodayStartupReminder(todayData)) {
    return;
  }
  app.globalData.hasShownTodayReminderThisSession = true;
  wx.setStorageSync("lastTodayReminderDate", todayDateText);
  this.setData({
    showTodayReminder: true,
    todayReminderData: todayData
  });
},
showTodayReminderManual() {
  const todayData = getTodayCoursesData();
  this.setData({
    showTodayReminder: true,
    todayReminderData: todayData
  });
},
closeTodayReminder() {
  this.setData({
    showTodayReminder: false
  });
},
goToTodayPage() {
  this.closeTodayReminder();
```

```
wx.switchTab({
  url: "/pages/today/today"
});
},
// 更多菜单
toggleMoreMenu() {
  this.setData({
    showMoreMenu: !this.data.showMoreMenu
  });
},
hideMoreMenu() {
  this.setData({
    showMoreMenu: false
  });
},
showDisclaimer() {
  this.hideMoreMenu();
  this.setData({
    showDisclaimerPopup: true
  });
},
closeDisclaimer() {
  this.setData({
    showDisclaimerPopup: false
  });
},
showNotice(event) {
  const type = event.currentTarget.dataset.type;
  const notice = type === "ticker" ? this.data.tickerNotice : this.data.homeNotice;
  if (!notice) return;
  this.setData({
    selectedNotice: notice,
    showNoticeDetail: true,
  });
},
closeNoticeDetail() {
  this.setData({
    showNoticeDetail: false,
    selectedNotice: null,
  });
},
dismissHomeNotice() {
  const notice = this.data.homeNotice;
  if (notice) {
    appConfigService.dismissNotice(notice);
  }
  this.setData({
    homeNotice: null,
  });
},
closeAppNoticeModal() {
  const notice = this.data.modalNotice;
  if (notice && notice.closable !== false) {
    appConfigService.dismissNotice(notice);
  }
  this.setData({
```

```

        showAppNoticeModal: false,
    });
},
onShareAppMessage() {
    const className = this.data.className;
    const title = className && className !== "未选课表" && className !== "请选择课表"
        ? `${className}的课程安排 · ${BRAND.appName}`
        : `${BRAND.appName} | 查看课程安排`;
    return {
        title: title,
        path: "/pages/index/index"
    };
},
noop() {},
});
// File: miniprogram/pages/schedule-view/schedule-view.js
const BRAND = require("../config/brand");
const { courseTimes } = require("../data/courseTimes");
const { buildScheduleColumns, normalizeCourse } = require("../utils/course");
const { getSettings, getCurrentScheduleTarget, saveSettings, setCurrentScheduleTarget } =
    require("../utils/storage");
const customCourseService = require("../services/customCourseService");
const releasePackService = require("../services/releasePackService");
const teachingCalendarService = require("../services/teachingCalendarService");
const { buildWeekPickerOptions } = require("../utils/weekPicker");
const {
    TOTAL_WEEKS,
    addLocalDays,
    clampWeek,
    formatDate,
    formatDateLabel,
    formatWeekRange,
    getCurrentTeachingWeek,
    getTodayTeachingInfo,
    getVisibleWeekdays,
    getWeekRangeByWeekNo,
} = require("../utils/week");
const SECTION_HEIGHT = 90;
const WEEKEND_DAY_WIDTH = 142;
const TIME_AXIS_WIDTH = 76;
function getContentWidthRpx() {
    return 750 - 32; // padding 左右各 16
}
function calendarChanged(left, right) {
    if (!left || !right) return Boolean(left || right);
    return left.term !== right.term ||
        left.releaseVersion !== right.releaseVersion ||
        JSON.stringify(left.termConfig || {}) !== JSON.stringify(right.termConfig || {}) ||
        JSON.stringify((left.weeks || []).map((week) => [week.weekNo, week.startDate, week.endDate])) !==
        JSON.stringify((right.weeks || []).map((week) => [week.weekNo, week.startDate, week.endDate]));
}
function getTypeText(type) {
    switch (type) {
        case "class": return "班级课表";
    }
}

```

```

    case "teacher": return "教师课表";
    case "classroom": return "教室课表";
    case "course": return "课程安排";
    default: return "课表";
  }
}
function safeDecodeURIComponent(value) {
  const text = String(value || "");
  try {
    return decodeURIComponent(text);
  } catch (error) {
    return text;
  }
}
function isTruthyParam(value) {
  return value === true || value === "1" || value === "true";
}
function parsePositiveIntParam(value) {
  const number = Number(value);
  return Number.isFinite(number) && number > 0 ? Math.floor(number) : null;
}
function getScheduleKindText(type, displayType, isAggregated) {
  if (type !== "class") {
    return getTypeText(type);
  }
  if (isAggregated || displayType === "major-schedule" || displayType === "major-shared-schedule") {
    return "专业聚合课表 · 暂未拆分行政班";
  }
  return "行政班级课表";
}
function getWeekdayText(weekday) {
  const map = ["", "周一", "周二", "周三", "周四", "周五", "周六", "周日"];
  return map[Number(weekday)] || "";
}
function getCourseOverview(courses) {
  return (courses || []).map(normalizeCourse).map((course, index) => Object.assign({}, course, {
    overviewKey: `${course.id || course.courseName || "course"}-${index}`,
    weekdayText: getWeekdayText(course.weekday),
    sectionText: `第${course.startSection}-${course.endSection}节`,
  }));
}
Page({
  data: {
    brand: BRAND,
    type: "class",
    typeText: "班级课表",
    name: "",
    title: "",
    semester: "",
    displayType: "",
    isAggregated: false,
    scheduleKindText: "行政班级课表",
    isCurrentTarget: false,
    allCourses: [],
  },

```

```

overviewCourses: [],
overviewExpanded: true,
hasCurrentWeekCourses: false,
hasVisibleWeekCourses: false,
visibleWeekCourseCount: 0,
currentWeekCourseCount: 0,
scheduleMeta: null,
currentWeek: 12,
totalWeeks: TOTAL_WEEKS,
weekRangeText: "",
weekScopeText: "周一至周五",
weekSwitcherLabel: "",
weekPickerOpen: false,
weekOptions: [],
sections: courseTimes,
sectionHeight: SECTION_HEIGHT,
scheduleHeight: courseTimes.length * SECTION_HEIGHT,
gridWidth: 718,
dayTrackWidth: 642,
dayColumnWidth: 128,
weekdays: [],
dayColumns: [],
showWeekend: true,
weekendShowMode: "overview",
detailVisible: false,
selectedCourse: null,
isFromShare: false,
},
onLoad(options) {
  const { type = "class", name = "", id = "", semester = "", term = "", releaseVersion =
"", displayType = "", isAggregated = "", shareScheduleId = "", week = "", weekday = "" }
= options;
  const decodedName = safeDecodeURIComponent(name);
  const decodedId = safeDecodeURIComponent(id);
  const immediateCalendar = teachingCalendarService.getImmediateActiveCalendar();
  const runtimeTerm = immediateCalendar.termConfig && immediateCalendar.termConfig.term
|| immediateCalendar.term || "";
  const decodedSemester = safeDecodeURIComponent(term || semester || runtimeTerm);
  const localActiveRelease = releasePackService.getLocalActiveRelease(decodedSemester);
  const decodedReleaseVersion = safeDecodeURIComponent(releaseVersion) ||
  (localActiveRelease && localActiveRelease.releaseVersion) ||
  "";
  const decodedDisplayType = safeDecodeURIComponent(displayType);
  const aggregated = isTruthyParam(isAggregated) || decodedDisplayType === "major-schedu
le" || decodedDisplayType === "major-shared-schedule";
  const title = decodedName || decodedId;
  const isFromShare = !!shareScheduleId;
  this._initialWeek = parsePositiveIntParam(week);
  this._initialWeekday = parsePositiveIntParam(weekday);
  this.setData({
    type,
    typeText: getTypeText(type),
    name: decodedName || decodedId,
    title,
    semester: decodedSemester,
    displayType: decodedDisplayType,

```

```

        isAggregated: aggregated,
        scheduleKindText: getScheduleKindText(type, decodedDisplayType, aggregated),
        isFromShare,
    });
    if (title) {
        wx.setNavigationBarTitle({
            title,
        });
    }
    // 检查是否已经是当前的绑定目标
    this.checkCurrentTargetStatus();
    // 通过 EventChannel 获取上一页传过来的课程数据
    let hasLoadedData = false;
    const eventChannel = this.getOpenerEventChannel();
    if (eventChannel && typeof eventChannel.on === "function") {
        eventChannel.on("acceptDataFromOpenerPage", (data) => {
            if (data && Array.isArray(data.courses)) {
                const hasUsableCourses = data.courses.length > 0;
                hasLoadedData = hasUsableCourses;
                const schedule = data.schedule || {};
                const nextDisplayType = schedule.displayType || this.data.displayType;
                const nextAggregated = Boolean(schedule.isAggregated || this.data.isAggregated |
| nextDisplayType === "major-schedule" || nextDisplayType === "major-shared-schedule");
                this.setData({
                    allCourses: data.courses,
                    displayType: nextDisplayType,
                    isAggregated: nextAggregated,
                    scheduleKindText: getScheduleKindText(this.data.type, nextDisplayType, nextAggregated),
                    scheduleMeta: schedule,
                }, () => {
                    if (hasUsableCourses) {
                        this.initScheduleLayout();
                    }
                });
            }
        });
    }
    // 降级与分享异步拉取处理
    setTimeout(() => {
        if (!hasLoadedData && (decodedName || decodedId)) {
            wx.showLoading({ title: "正在拉取课表..." });
            const loadByIndexedId = Boolean(decodedId);
            if (loadByIndexedId) {
                if (!type || !decodedId || !decodedReleaseVersion) {
                    wx.hideLoading();
                    this.showScheduleOpenError();
                    this.initScheduleLayout();
                    return;
                }
                releasePackService.loadDetail(type, decodedId, {
                    term: decodedSemester,
                    releaseVersion: decodedReleaseVersion,
                }, {
                    forceNetwork: true,
                    timeout: 15000,

```

```

        retries: 1,
    })
    .then((res) => {
        wx.hideLoading();
        const schedule = res.schedule || res.detail || {};
        const courses = Array.isArray(schedule.courses) ? schedule.courses : [];
        if (!res || res.success === false || !courses.length) {
            this.showScheduleOpenError({ type });
            this.initScheduleLayout();
            return;
        }
        this.setData({
            name: decodedName || schedule.className || schedule.teacherName || schedule.
e.roomName || schedule.courseName || decodedId,
            title: decodedName || schedule.className || schedule.teacherName || schedu
le.roomName || schedule.courseName || decodedId,
            allCourses: courses,
            scheduleMeta: Object.assign({}, schedule, {
                scheduleVersion: res.releaseVersion || res.version || decodedReleaseVers
ion,
            }),
        }, () => {
            this.initScheduleLayout();
        });
    })
    .catch((err) => {
        wx.hideLoading();
        console.error("按索引拉取课表失败", err);
        if (type === "classroom" && decodedName) {
            this.openResolvedClassroomSchedule(decodedName, decodedSemester, decodedRe
leaseVersion, err);
            return;
        }
        this.showScheduleOpenError({ type });
        this.initScheduleLayout();
    });
    return;
}
const request = require("../utils/request");
let apiUrl = "/api/fosu/class-schedule";
let requestParams = {
    semester: decodedSemester,
};
if (type === "teacher") {
    apiUrl = "/api/fosu/teacher-schedule";
    requestParams.keyword = decodedName;
} else if (type === "classroom") {
    apiUrl = "/api/fosu/classroom-schedule";
    requestParams.classroomName = decodedName;
} else if (type === "course") {
    apiUrl = "/api/fosu/course-schedule";
    requestParams.courseName = decodedName;
} else {
    apiUrl = "/api/fosu/class-schedule";
    requestParams.className = decodedName;
}

```

```

    request.post(apiUrl, requestParams, { showLoading: false, silentError: true })
      .then((res) => {
        wx.hideLoading();
        let courses = [];
        let scheduleMeta = null;
        if (res && res.success) {
          if (type === "class" && Array.isArray(res.classes) && res.classes.length > 0
) {
            courses = res.classes[0].courses || [];
            scheduleMeta = res.classes[0];
          } else if (type === "teacher" && Array.isArray(res.teachers) && res.teachers
.length > 0) {
            courses = res.teachers[0].courses || [];
          } else if (type === "classroom" && Array.isArray(res.classrooms) && res.clas
srooms.length > 0) {
            courses = res.classrooms[0].courses || [];
          } else if (type === "course" && Array.isArray(res.coursesList) && res.cour
seList.length > 0) {
            courses = res.coursesList[0].courses || [];
          }
        }
        this.setData({
          allCourses: courses,
          scheduleMeta: scheduleMeta || this.data.scheduleMeta,
        }, () => {
          this.initScheduleLayout();
        });
      })
      .catch((err) => {
        wx.hideLoading();
        console.error("异步拉取课表失败", err);
        this.initScheduleLayout();
      });
    } else if (!hasLoadedData) {
      this.initScheduleLayout();
    }
  }, 300);
},
checkCurrentTargetStatus() {
  const currentTarget = getCurrentScheduleTarget();
  const meta = this.data.scheduleMeta || {};
  const currentId = currentTarget && (currentTarget.detailId || currentTarget.id || curr
entTarget.classId || "");
  const detailId = meta.detailId || meta.id || meta.scheduleId || meta.classId || "";
  const isCurrent = currentTarget &&
    currentTarget.type === this.data.type &&
    (currentId && detailId ? currentId === detailId : currentTarget.name
=== this.data.name) &&
    (currentTarget.term || currentTarget.semester) === this.data.semeste
r;
  this.setData({
    isCurrentTarget: Boolean(isCurrent),
  });
},
openResolvedClassroomSchedule(roomName, term, releaseVersion, originalError) {
  wx.showLoading({ title: "正在重新定位教室..." });
}

```

```

    releasePackService.resolveClassroomDetail(roomName, {
      term,
      releaseVersion,
    }, {
      timeout: 15000,
      retries: 1,
    })
    .then((res) => {
      wx.hideLoading();
      const schedule = res.schedule || res.detail || {};
      const courses = Array.isArray(schedule.courses) ? schedule.courses : [];
      if (!courses.length) {
        this.showScheduleOpenError({ type: "classroom" });
        this.initScheduleLayout();
        return;
      }
      this.setData({
        name: schedule.roomName || schedule.classroomName || roomName,
        title: schedule.roomName || schedule.classroomName || roomName,
        allCourses: courses,
        scheduleMeta: Object.assign({}, schedule, {
          id: res.detailId || res.resolvedId || schedule.id || "",
          detailId: res.detailId || res.resolvedId || schedule.id || "",
          scheduleVersion: res.releaseVersion || res.version || releaseVersion,
        }),
      }, () => {
        this.initScheduleLayout();
      });
    })
    .catch((error) => {
      wx.hideLoading();
      console.error("按教室名解析课表失败", error, originalError);
      this.showScheduleOpenError({ type: "classroom" });
      this.initScheduleLayout();
    });
  },
  showScheduleOpenError(options = {}) {
    const isClassroom = options.type === "classroom" || this.data.type === "classroom";
    wx.showModal({
      title: "未找到课表",
      content: isClassroom
        ? "该教室暂无课表详情，但空闲结果仍可参考。"
        : "该链接对应的课表不存在或已被新版本替换，可返回全校搜索重新查找。",
      confirmText: "去全校",
      cancelText: "留在此页",
      success: (res) => {
        if (res.confirm) {
          wx.switchTab({
            url: "/pages/school/school",
          });
        }
      },
    });
  },
  initScheduleLayout() {
    const settings = getSettings();
  }
}

```

```

    const calendar = teachingCalendarService.getImmediateActiveCalendar({ term: this.data.semester });
    const termConfig = calendar.termConfig || {};
    const now = new Date();
    const currentWeek = this._initialWeek
      ? clampWeek(this._initialWeek, termConfig)
      : settings.manualWeekOverride
      ? clampWeek(settings.currentWeek, termConfig)
      : getCurrentTeachingWeek(now, calendar.weeks || [], termConfig);
    const showWeekend = this._initialWeekday >= 6 ? true : (settings.showWeekend || false);
;
    this.activeTeachingCalendar = calendar;
    this.setData({
      currentWeek,
      totalWeeks: termConfig.totalWeeks || TOTAL_WEEKS,
      showWeekend,
      weekendShowMode: settings.weekendShowMode || "overview",
    }, () => {
      this.renderSchedule();
    });
    teachingCalendarService.loadActiveTeachingCalendar({ term: this.data.semester })
      .then((latest) => {
        if (!calendarChanged(this.activeTeachingCalendar, latest)) return;
        this.activeTeachingCalendar = latest;
        const latestConfig = latest.termConfig || {};
        const nextWeek = this._initialWeek
          ? clampWeek(this._initialWeek, latestConfig)
          : getSettings().manualWeekOverride
          ? clampWeek(getSettings().currentWeek, latestConfig)
          : getCurrentTeachingWeek(new Date(), latest.weeks || [], latestConfig);
        this.setData({
          currentWeek: nextWeek,
          totalWeeks: latestConfig.totalWeeks || TOTAL_WEEKS,
        }, () => this.renderSchedule());
      })
      .catch(() => {});
  },
  renderSchedule() {
    const settings = getSettings();
    const calendar = this.activeTeachingCalendar || teachingCalendarService.getImmediateActiveCalendar({ term: this.data.semester });
    const termConfig = calendar.termConfig || {};
    const now = new Date();
    const todayInfo = getTodayTeachingInfo(now, calendar.weeks || [], termConfig);
    const currentWeek = this.data.currentWeek;
    const weekInfo = getWeekRangeByWeekNo(currentWeek, calendar.weeks || [], termConfig);
    const showWeekend = this.data.showWeekend;
    const weekendShowMode = this.data.weekendShowMode || "overview";
    const baseWeekdays = getVisibleWeekdays(showWeekend, now);
    const weekdays = baseWeekdays.map((day, index) => {
      const date = addLocalDays(weekInfo.startDate, index);
      const dateInfo = getTodayTeachingInfo(date, calendar.weeks || [], termConfig);
      return Object.assign({}, day, {
        date: formatDate(date),
        dateLabel: formatDateLabel(date),
        isToday: currentWeek === todayInfo.rawWeekNo && day.weekday === todayInfo.physical
      });
    });
  }
}

```

```

Weekday,
  isTeachingDay: dateInfo.isTeachingDay,
  scheduleWeek: dateInfo.weekNo,
  scheduleWeekday: dateInfo.weekday,
  teachingEventType: dateInfo.teachingEventType,
  teachingEventNote: dateInfo.teachingEventNote,
});
});
const courses = this.data.allCourses.map(normalizeCourse);
const meta = this.data.scheduleMeta || {};
const dayColumns = buildScheduleColumns(courses, weekdays, currentWeek, {
  sectionHeight: SECTION_HEIGHT,
  hideInactiveCourses: settings.hideInactiveCourses,
  normalized: true,
  targetType: this.data.type,
  targetId: meta.detailId || meta.id || meta.classId || this.data.name || "",
  targetName: this.data.name,
  semester: this.data.semester,
  releaseVersion: meta.scheduleVersion || meta.releaseVersion || "",
});
const currentWeekCourseCount = dayColumns.reduce((total, day) => total + (day.activeCourseCount || 0), 0);
const visibleWeekCourseCount = dayColumns.reduce((total, day) => total + (day.visibleCourseCount || 0), 0);
const contentWidth = getContentWidthRpx();
let dayColumnWidth = 128;
let scrollX = false;
if (showWeekend) {
  if (weekendShowMode === "detail") {
    dayColumnWidth = WEEKEND_DAY_WIDTH; // 142
    scrollX = true;
  } else {
    // 七天概览模式，一屏显示周一至周日，无横滚
    dayColumnWidth = Math.floor((contentWidth - TIME_AXIS_WIDTH) / 7); // (750 - 32 -
76) / 7 = 91
    scrollX = false;
  }
} else {
  // 五天详细，一屏无横滚
  dayColumnWidth = Math.floor((contentWidth - TIME_AXIS_WIDTH) / 5); // 128
  scrollX = false;
}
const dayTrackWidth = dayColumnWidth * weekdays.length;
const gridWidth = TIME_AXIS_WIDTH + dayTrackWidth;
const weekRangeText = formatWeekRange(weekInfo.startDate, weekInfo.endDate);
const weekSwitcherLabel = weekRangeText ? `${weekRangeText} · 第${currentWeek}周` : `
日期待同步 · 第${currentWeek}周`;
this.setData({
  weekRangeText,
  weekScopeText: showWeekend ? "周一至周日" : "周一至周五",
  weekSwitcherLabel,
  weekOptions: buildWeekPickerOptions(calendar),
  gridWidth,
  dayTrackWidth,
  dayColumnWidth,
  weekdays,

```

```

    dayColumns,
    scrollX,
    weekendShowMode,
    overviewCourses: getCourseOverview(this.data.allCourses),
    hasCurrentWeekCourses: currentWeekCourseCount > 0,
    hasVisibleWeekCourses: visibleWeekCourseCount > 0,
    currentWeekCourseCount,
    visibleWeekCourseCount,
  });
},
onWeekChange(event) {
  const detail = event && event.detail || {};
  const type = detail.type;
  if (type !== "prev" && type !== "next" && type !== "current" && type !== "select") return;

  const calendar = this.activeTeachingCalendar || teachingCalendarService.getImmediateActiveCalendar({ term: this.data.semester });
  const termConfig = calendar.termConfig || {};
  const nextWeek = type === "current"
    ? getCurrentTeachingWeek(new Date(), calendar.weeks || [], termConfig)
    : clampWeek(detail.week, termConfig);
  if (type !== "current" && nextWeek === this.data.currentWeek) return;
  this._initialWeek = null;
  saveSettings({
    currentWeek: nextWeek,
    manualWeekOverride: type !== "current",
  });
  this.setData({
    currentWeek: nextWeek,
  }, () => {
    this.renderSchedule();
  });
},
onWeekPickerModalChange(event) {
  this.setData({ weekPickerOpen: Boolean(event.detail && event.detail.visible) });
},
toggleBindTarget() {
  if (this.data.isCurrentTarget) {
    wx.showToast({
      title: "已是当前首页课表",
      icon: "none",
    });
    return;
  }
  const nowStr = new Date().toLocaleTimeString("zh-CN", { hour12: false, hour: "2-digit", minute: "2-digit" });
  const meta = this.data.scheduleMeta || {};
  const releaseVersion = meta.scheduleVersion || meta.releaseVersion || meta.version || "";
  const detailId = meta.detailId || meta.id || meta.scheduleId || meta.classId || this.data.name || "";
  const target = {
    type: this.data.type,
    id: meta.id || detailId,
    detailId,
    name: this.data.name,
  }

```

```
    term: this.data.semester,
    semester: this.data.semester,
    courses: this.data.allCourses,
    updateTime: nowStr,
    classId: meta.classId || "",
    className: meta.className || this.data.name || "",
    displayType: meta.displayType || this.data.displayType || "",
    isAggregated: this.data.isAggregated,
    releaseVersion,
    source: "schedule-view",
  };
  if (!setCurrentScheduleTarget(target)) {
    wx.showModal({
      title: "设置失败",
      content: "本地课表保存失败，请稍后重试。现有课表不会被清除。",
      showCancel: false,
    });
    return;
  }
  this.setData({
    isCurrentTarget: true,
  });
  wx.showToast({
    title: "绑定首页成功",
    icon: "success",
    duration: 1200,
  });
  setTimeout(() => {
    wx.switchTab({
      url: "/pages/index/index",
    });
  }, 1000);
},
onCourseTap(event) {
  this.setData({
    selectedCourse: event.detail.course,
    detailVisible: true,
  });
},
toggleOverview() {
  this.setData({
    overviewExpanded: !this.data.overviewExpanded,
  });
},
onOverviewCourseTap(event) {
  const index = Number(event.currentTarget.dataset.index);
  const course = this.data.overviewCourses[index];
  if (!course) return;
  this.setData({
    selectedCourse: course,
    detailVisible: true,
  });
},
closeCourseDetail() {
  this.setData({
    selectedCourse: null,
```

```

        detailVisible: false,
    });
},
onCopyCourseToCustom(event) {
    try {
        customCourseService.saveCustomCourseDraft(event.detail.course || this.data.selectedCourse);
        this.closeCourseDetail();
        wx.navigateTo({
            url: "/pages/custom-courses/custom-courses",
        });
    } catch (error) {
        wx.showToast({
            title: "课程信息不完整",
            icon: "none",
        });
    }
},
onShareAppMessage() {
    const meta = this.data.scheduleMeta || {};
    const detailId = meta.detailId || meta.id || meta.classId || this.data.name || "";
    const releaseVersion = meta.scheduleVersion || meta.releaseVersion || "";
    return {
        title: `${this.data.name}的课程安排 · ${BRAND.appName}`,
        path: `/pages/schedule-view/schedule-view?shareScheduleId=${encodeURIComponent(meta.classId || this.data.name)}&id=${encodeURIComponent(detailId)}&name=${encodeURIComponent(this.data.name)}&type=${this.data.type}&semester=${encodeURIComponent(this.data.semester)}&term=${encodeURIComponent(this.data.semester)}&releaseVersion=${encodeURIComponent(releaseVersion)}&displayType=${encodeURIComponent(this.data.displayType || "")}&isAggregated=${this.data.isAggregated ? "1" : "0"}&preview=1`
    };
}
});
// File: miniprogram/pages/today/today.js
const { getTodayCoursesData } = require("../utils/todayReminder");
const { getSettings } = require("../utils/storage");
const appConfigService = require("../services/appConfigService");
const currentScheduleService = require("../services/currentScheduleService");
const customCourseService = require("../services/customCourseService");
const courseOverrideService = require("../services/courseOverrideService");
const BRAND = require("../config/brand");
const classroomSearch = require("../utils/classroomSearch");
const AI_PENDING_TODAY_QUERY_KEY = "FOSU_AI_PENDING_TODAY_QUERY";
Page({
    data: {
        brand: BRAND,
        dateText: "",
        weekdayText: "",
        className: "未选择当前课表",
        currentWeek: 12,
        todayTitle: "",
        termPhase: "unknown",
        dataSourceText: "课程数据 · 本地缓存",
        courseCountText: "今日共 0 门课",
        courses: [],
        appConfig: { notices: [] },
    },

```

```

    urgentNotice: null,
    dataVersionText: "",
    selectedCourse: null,
    detailVisible: false,
    nextBuildingCode: "",
    emptyTitle: "今天没有课程, 好好休息",
    emptyDesc: "这里会根据当前班级、教学周和星期自动筛选课程。"
  },
  onShow() {
    const pending = this.consumeAiPendingTodayQuery();
    this.loadToday();
    this.loadPageConfig();
    this.refreshCurrentTargetSilently();
    if (pending) {
      this.applyAiPendingTodayQuery(pending);
    }
  },
  refreshCurrentTargetSilently() {
    currentScheduleService.ensureCurrentScheduleFresh({
      silent: true,
      notify: true,
    }).then((result) => {
      if (result && result.status === "UPDATED") {
        this.loadToday();
        if (result.shouldNotify) {
          wx.showToast({
            title: "课表已更新至最新数据",
            icon: "none",
            duration: 1200,
          });
        }
      }
    }).catch(() => {});
  },
  consumeAiPendingTodayQuery() {
    let query = null;
    try {
      query = wx.getStorageSync(AI_PENDING_TODAY_QUERY_KEY);
      if (query) {
        wx.removeStorageSync(AI_PENDING_TODAY_QUERY_KEY);
      }
    } catch (error) {
      query = null;
    }
    if (!query || typeof query !== "object" || Array.isArray(query)) return null;
    return query;
  },
  applyAiPendingTodayQuery(query = {}) {
    const action = String(query.action || "focus").toLowerCase();
    if (action === "refresh") {
      this.loadToday();
      this.loadPageConfig();
    }
    wx.showToast({
      title: "已根据小序建议打开今日安排",
      icon: "none",
    });
  }
}

```

```

    });
  },
  loadPageConfig() {
    return appConfigService.loadAppConfig()
      .then((config) => {
        const normalizedConfig = Object.assign({
          notices: [],
          urgentNotice: null,
          banners: [],
          appConfig: {},
        }, config || {});
        if (!Array.isArray(normalizedConfig.notices)) normalizedConfig.notices = [];
        if (!Array.isArray(normalizedConfig.banners)) normalizedConfig.banners = [];
        if (normalizedConfig.urgentNotice === undefined) normalizedConfig.urgentNotice = null;

        const urgentNotice = appConfigService.getPageNotices(normalizedConfig, "today")
          .find((notice) => notice.priority === "urgent" && notice.displayMode !== "ticker") || null;

        const latestUpdatedAt = appConfigService.getLatestDataUpdatedAt(normalizedConfig);
        this.setData({
          appConfig: normalizedConfig,
          urgentNotice,
          dataVersionText: latestUpdatedAt ? `数据更新于 ${appConfigService.formatConfigTime(latestUpdatedAt)}` : "",
        });
      })
      .catch((err) => {
        console.warn("今日页公告配置加载失败", err);
        this.setData({
          appConfig: appConfigService.normalizeConfig
            ? appConfigService.normalizeConfig(this.data.appConfig)
            : Object.assign({ notices: [], banners: [], appConfig: {} }, this.data.appConfig || {}),
          urgentNotice: null,
        });
      });
  },
  loadToday() {
    const data = getTodayCoursesData();
    const { hasSchedule, dateText, weekdayText, className, currentWeek, courses, termPhase, termPhaseText } = data;

    const todayTitle = termPhase === "in-term"
      ? `${weekdayText} · 第${currentWeek}周`
      : `${weekdayText} · ${termPhaseText || "教学周待同步"}`;

    const settings = getSettings();
    const { getCourseDataSource } = require("../utils/course");
    const dataSource = getCourseDataSource();
    let displayClassName = className;
    const { getCurrentScheduleTarget } = require("../utils/storage");
    const target = getCurrentScheduleTarget();
    if (target) {
      displayClassName = target.type === "teacher"
        ? `${target.name} 老师`
        : (target.type === "classroom" ? `${target.name} 教室` : target.name);
    }
    if (!hasSchedule) {

```

```

        this.setData({
            dateText,
            weekdayText,
            className: displayClassName || "未选择当前课表",
            currentWeek,
            todayTitle,
            termPhase,
            dataSourceText: "未绑定课表",
            courseCountText: "今日共 0 门课",
            courses: [],
            nextBuildingCode: "",
            emptyTitle: "未绑定当前课表",
            emptyDesc: "请先前往「全校」页面查找班级，并在课表详情页点击「设为当前」进行绑定。
"
        });
        return;
    }
    let emptyTitle = "今天没有课程，好好休息";
    let emptyDesc = "这里会根据当前班级、教学周和星期自动筛选课程。";
    let courseCountText = `今日共 ${courses.length} 门课`;
    if (termPhase === "before-term") {
        emptyTitle = "当前处于开学前";
        emptyDesc = "课表已保留，开学后会按照教学周历自动显示当天课程。";
        courseCountText = "尚未开学";
    } else if (termPhase === "after-term") {
        emptyTitle = "本学期已结束";
        emptyDesc = "当前处于寒暑假或学期间隔，课表数据仍会保留。";
        courseCountText = "假期模式";
    } else if (termPhase === "unknown") {
        emptyTitle = "教学周数据待同步";
        emptyDesc = "已暂停按星期推算课程，避免显示错误的今日课表。";
        courseCountText = "教学周待同步";
    }
    this.setData({
        dateText,
        weekdayText,
        todayTitle,
        termPhase,
        className: displayClassName,
        currentWeek,
        dataSourceText: dataSource.text,
        courseCountText,
        courses,
        nextBuildingCode: this.resolveNextBuildingCode(courses),
        emptyTitle,
        emptyDesc
    });
},
resolveNextBuildingCode(courses) {
    const list = Array.isArray(courses) ? courses : [];
    for (const course of list) {
        const classroom = course && (course.displayClassroom || course.canonicalClassroom ||
course.classroom || course.roomName);
        const parsed = classroomSearch.parseClassroomQuery(classroom || "");
        if (parsed.buildingCode) return parsed.buildingCode;
    }
}

```

```

        return "";
    },
    onCourseTap(event) {
        this.setData({
            selectedCourse: event.detail.course,
            detailVisible: true,
        });
    },
    closeCourseDetail() {
        this.setData({
            selectedCourse: null,
            detailVisible: false,
        });
    },
    onEditExistingCourse(event) {
        try {
            courseOverrideService.saveEditDraft(event.detail.course || this.data.selectedCourse)
;
            this.closeCourseDetail();
            wx.navigateTo({ url: "/pages/custom-courses/custom-courses" });
        } catch (error) {
            wx.showToast({ title: "请从个性化页面选择课程", icon: "none" });
        }
    },
    goEmptyRoom() {
        wx.navigateTo({
            url: "/pages/empty-room/empty-room",
        });
    },
    goCampusMap() {
        const code = this.data.nextBuildingCode || "";
        wx.navigateTo({
            url: ` /packageMaps/pages/campus-map/campus-map${code} ? `?q=${encodeURIComponent(code
)}}` : ""}`,
        });
    },
    goAiAssistant() {
        wx.navigateTo({
            url: ` /packageXiaofu/pages/ai-assistant/ai-assistant?q=${encodeURIComponent(" 查询今
日安排")}`,
        });
    },
    onCopyCourseToCustom(event) {
        try {
            customCourseService.saveCustomCourseDraft(event.detail.course || this.data.selectedC
ourse);
            this.closeCourseDetail();
            wx.navigateTo({
                url: "/pages/custom-courses/custom-courses",
            });
        } catch (error) {
            wx.showToast({
                title: "课程信息不完整",
                icon: "none",
            });
        }
    }
}

```

```
},  
});
```